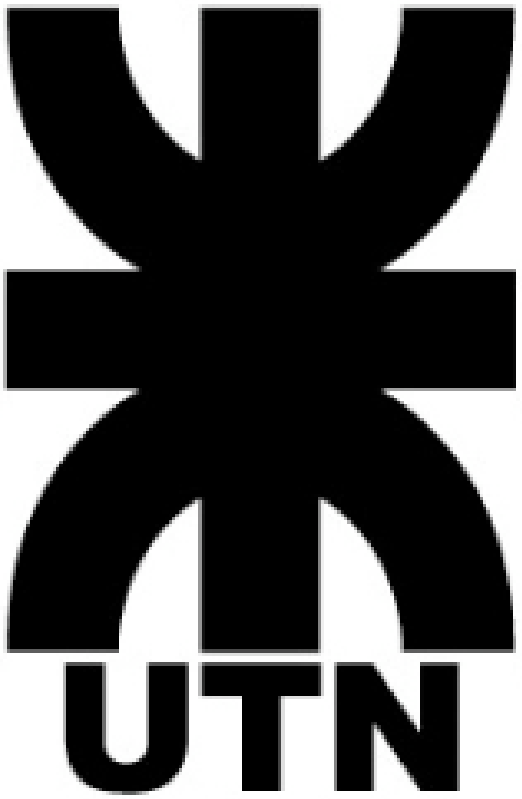
RESUMEN SEGUNDO PARCIAL

Ingeniería y Calidad de Software (ISW)



Unidades Comprendidas

- **Unidad 2:** Gestión Lean-Agile de Productos de Software
- **Unidad 3:** Gestión de Software como Producto
- **Unidad 4:** Aseguramiento de Calidad de Proceso y de Producto

Autor: Fanfliet, Juan Ignacio

Unidad 2: Gestión Lean Agile de Productos de Software

Filosofia Lean

Lean es una filosofía de gestión, originada en el sistema de producción de Toyota, que busca **maximizar el valor entregado al cliente y reducir el desperdicio**. En desarrollo de software, implica concentrar el trabajo en resolver necesidades reales y mejorar el proceso completo de forma continua, desde que surge una necesidad hasta que el usuario recibe una solución.

El **valor** se determina desde la perspectiva del cliente: una actividad aporta valor cuando contribuye a obtener un producto que satisface sus necesidades. El **desperdicio** es el consumo de tiempo, esfuerzo o recursos que no contribuye a ese resultado.

Lean busca mejorar el **flujo de trabajo** reduciendo esperas, acumulaciones y retrabajo. Para ello, observa y mejora el proceso completo, evitando optimizaciones locales que perjudiquen al conjunto, con el objetivo de entregar valor al cliente lo antes posible.

Relacion Lean - Agile

Lean y Agile son compatibles: ambos buscan entregar valor al cliente, aprender mediante retroalimentación y mejorar continuamente. Lean pone especial atención en eliminar desperdicios y optimizar el flujo completo; Agile enfatiza la adaptación al cambio y la entrega frecuente de software funcionando. Los principios Lean se reflejan en prácticas ágiles como el refinamiento progresivo, los equipos autogestionados, la calidad incorporada y las entregas pequeñas y frecuentes.

Los siete principios de Lean aplicados al software

1. Eliminar desperdicios. Consiste en identificar y reducir actividades que consumen recursos sin aportar valor. Por ejemplo, desarrollar funcionalidades innecesarias, mantener trabajo incompleto o duplicar documentación. Antes de eliminar algo, hay que comprender su utilidad: las actividades necesarias se simplifican; las innecesarias se eliminan.

2. Amplificar el aprendizaje. El desarrollo de software requiere descubrir necesidades y encontrar soluciones. Lean propone aprender mediante ciclos cortos, feedback, pruebas y experimentos, compartiendo el conocimiento dentro del equipo. Se relaciona con la transparencia y el conocimiento compartido: la información necesaria para decidir no debe quedar concentrada en una sola persona.

3. Embeber la integridad conceptual. Incorporar la calidad desde el inicio y durante todo el desarrollo, asegurando que las partes del producto funcionen de manera coherente y respondan a las necesidades del usuario. Se logra mediante buen diseño, integración frecuente, pruebas y refactorización.

4. Diferir compromisos hasta el último momento responsable. Consiste en postergar decisiones difíciles de revertir mientras todavía resulte conveniente obtener más información. Se mantienen alternativas abiertas y se decide antes de que la demora haga perder opciones o genere perjuicios. No significa dejar todo para último momento.

5. Dar poder al equipo. Brindar al equipo la autonomía, la información y la formación necesarias para decidir cómo realizar su trabajo y responsabilizarse por los resultados. Los líderes orientan, acompañan y eliminan obstáculos.

6. Ver el todo. Propone optimizar el proceso completo y el resultado para el cliente, evitando mejoras locales que perjudiquen al sistema. Por ejemplo, aumentar la cantidad de funcionalidades programadas puede ser contraproducente si supera la capacidad de pruebas y acumula trabajo sin terminar.

7. Entregar lo antes posible. Trabajar en lotes pequeños y reducir demoras permite obtener valor y retroalimentación tempranamente. Las entregas frecuentes también facilitan detectar problemas y adaptarse a cambios. La rapidez se consigue mejorando el flujo y reduciendo desperdicios, manteniendo la calidad.

Tipos de desperdicio

La clasificación utilizada en la filmina distingue:

- **Tipo 1 — Desperdicio necesario:** no agrega valor directamente para el cliente, pero resulta necesario bajo las condiciones actuales. Se busca **minimizarlo** y encontrar formas más eficientes de realizarlo. Por ejemplo, una carga administrativa obligatoria que podría simplificarse o automatizarse.
- **Tipo 2 — Puro desperdicio:** no agrega valor ni es necesario. Se busca **evitarlo o eliminarlo**. Por ejemplo, registrar la misma información en dos lugares cuando uno de ellos no tiene ninguna utilidad.

La distinción exige analizar el contexto. Los Poppendieck señalan que la documentación puede ser desperdicio cuando nadie la necesita, pero también puede aportar valor cuando ayuda a

comprender, construir, probar o cumplir requisitos relevantes. **No toda actividad de soporte es desperdicio.**

Los siete desperdicios en el desarrollo de software

1. Trabajo parcialmente terminado — Inventario. Es trabajo iniciado que todavía no puede generar el valor esperado: requisitos sin implementar, código sin integrar, funcionalidades sin probar o cambios pendientes de entrega. Acumula inversión sin obtener resultados y puede volverse obsoleto. Se reduce trabajando en lotes pequeños y terminando antes de comenzar más.

2. Características extra — Sobreproducción. Son funcionalidades desarrolladas sin una necesidad real que las justifique. Además del esfuerzo inicial, requieren pruebas, mantenimiento y aumentan la complejidad. Por ejemplo, agregar opciones "por si algún día se necesitan", sin evidencia de su utilidad. Se relaciona con **YAGNI —You Aren't Gonna Need It—**: evitar construir funcionalidades para necesidades hipotéticas que todavía no están justificadas.

3. Procesos extra — Sobreprocesamiento. Son pasos extra, documentos o controles que no aportan valor suficiente. Por ejemplo, repetir aprobaciones o elaborar informes que nadie utiliza. Se debe revisar su propósito y simplificarlos, conservando los controles necesarios.

4. Transporte - Cambio de tareas / Transferencia de conocimiento. Son transferencias de trabajo o información entre personas y áreas que generan demoras, pérdida de contexto o necesidad de volver a explicar lo mismo. Por ejemplo, pasar un requerimiento por múltiples intermediarios antes de que llegue a quien lo implementa. Compartir conocimiento útil no es desperdicio; lo es el esfuerzo innecesario o la pérdida de información producida por esos traspasos.

5. Esperas o demoras. Son períodos en los que el trabajo no avanza porque falta información, una aprobación, un entorno o el resultado de otra actividad. Retrasan la entrega de valor al cliente. Por ejemplo, una funcionalidad terminada que espera varios días para ser probada.

6. Movimiento y búsqueda de información. Es el esfuerzo innecesario para localizar información, acceder a herramientas o recuperar la concentración al alternar entre tareas y proyectos. Por ejemplo, trabajar parte del día en un proyecto y luego en otro exige reconstruir el contexto antes de continuar, lo que consume tiempo y puede favorecer errores.

7. Defectos. Son problemas que obligan a investigar, corregir y repetir actividades. Su impacto puede crecer cuanto más tiempo permanecen sin detectarse. Una regla de negocio mal interpretada puede afectar código, pruebas y documentación. La prevención, las pruebas tempranas y la integración frecuente reducen este desperdicio.

8. Talento no utilizado. Consiste en desaprovechar conocimientos, capacidades e ideas de las personas. Por ejemplo, ofrecer únicamente una carrera hacia puestos de dirección a un buen programador que desea crecer técnicamente puede hacer perder un buen desarrollador y obtener un líder poco motivado o preparado.

Método KANBAN

Kanban es un método para organizar, gestionar y mejorar el flujo de trabajo en actividades que requieren conocimientos y toma de decisiones, como el desarrollo de software. Busca equilibrar la demanda con la capacidad disponible para entregar valor de forma continua. Parte del proceso existente y propone mejoras graduales, respetando inicialmente los roles y las responsabilidades actuales.

Para lograrlo, permite **visualizar el trabajo, limitar la cantidad de tareas iniciadas y aún no terminadas —WIP—, establecer reglas claras y utilizar métricas para identificar problemas y evaluar mejoras**.

Kanban es más que un tablero: además de mostrar el trabajo, regula su incorporación según la capacidad disponible. Cuando se libera un lugar y se cumplen las reglas acordadas, se puede tomar una nueva tarea. Así se busca reducir las acumulaciones, las esperas y la multitarea, favoreciendo terminar trabajos antes de comenzar otros.

Origen: Kanban y Just in Time

El término oriental *kanban* alude a una señal visual, tarjeta o letrero indicador. Sus raíces industriales remiten directamente al esquema de producción implementado por Toyota, donde dichos avisos regulaban el reabastecimiento limitando las existencias en curso.

Esta visión se enlaza con el paradigma **Just in Time**, tendiente a elaborar únicamente lo indispensable en el momento oportuno, anulando acumulaciones inconclusas. En el terreno informático, la dinámica se ajusta posibilitando tomar nuevo trabajo tan pronto como se libere espacio operativo. Las tarjetas simbolizan entregables concretos, mientras los umbrales prefijados gobiernan su incorporación.

Valores de Kanban

Los principios axiológicos del esquema orientan la dinámica de colaboración e implementación:

- **Transparencia:** exhibir abiertamente datos del trabajo, reglas y restricciones para respaldar resoluciones acertadas.
- **Equilibrio:** equiparar demandas externas con la capacidad real y requerimientos diversos para prevenir el colapso del equipo.
- **Colaboración:** unir esfuerzos colectivos enfocado en optimizar el servicio.
- **Foco en el cliente:** alinear las decisiones del sistema con las necesidades y el valor real percibido por el destinatario.
- **Flujo:** comprender de manera completa el recorrido de las tareas hasta materializarse en entregas de valor.
- **Liderazgo:** promover iniciativas de mejora desde todos los niveles de la organización.
- **Entendimiento:** analizar la realidad actual en profundidad antes de procurar transformaciones.
- **Acuerdo:** comprometerse conjuntamente con la mejora, considerando las diferencias de opinión.
- **Respeto:** valorar los aportes, la trayectoria y el rol de cada participante.

Principios de Kanban

Principios de gestión del cambio

Comenzar con lo que se hace actualmente. Supone entender los procesos actuales tal y como están siendo realizados en la actualidad, y respetar los roles y responsabilidades actuales de cada persona.

Acordar la mejora mediante el cambio evolutivo. Implica instaurar ajustes graduales examinando los impactos reales, de forma tal que la dinámica progrese basada en evidencia concreta.

Fomentar actos de liderazgo en todos los niveles. Todo integrante posee la facultad de detectar ineficiencias y articular sugerencias, posibilitando que la iniciativa surja tanto de la base operativa como de la línea directiva. No se concentra el poder en una sola persona.

Principios de entrega de servicios

Comprender y atender las necesidades y expectativas del cliente. Conocer qué necesita y espera el cliente, y orientar el servicio para satisfacerlo.

Gestionar el trabajo y permitir la autoorganización de las personas. Seguir el avance de las tareas y detectar obstáculos, dejando que las personas se organicen para resolverlos y completar el trabajo.

Revisar y evolucionar las políticas del servicio. Evaluar y ajustar las reglas de trabajo para mejorar el servicio completo, evitando beneficiar a una parte a costa de perjudicar a las demás.

Prácticas generales de Kanban

1. Visualizar el trabajo

Consiste en esquematizar el recorrido de las tareas a través de un panel donde las columnas reflejan las fases operativas y las tarjetas representan los elementos de trabajo requeridos. Tal esquema permite explicitar actividades pendientes, esfuerzos en curso, demoras o bloqueos, consolidando un entendimiento alineado que potencia la coordinación.

El tablero representa el proceso real del servicio, por lo que sus columnas se adaptan al contexto. Puede distinguir etapas de trabajo activo y colas de espera, como “Desarrollo en progreso” y “Listo para pruebas”. Las tarjetas representan tipos de trabajo definidos, como funcionalidades, defectos o mantenimiento que van pasando por cada etapa.

Para favorecer la finalización, se suele revisar de derecha a izquierda: primero se observa qué falta para terminar o entregar los trabajos más avanzados y luego qué puede iniciarse. La idea es **dejar de empezar y empezar a terminar**.

2. Limitar el trabajo en progreso - WIP

El **WIP** es la cantidad de ítems iniciados y todavía no terminados dentro del tramo del proceso que se está midiendo. Incluye los que están siendo trabajados, esperando o bloqueados. El **límite WIP** es el máximo acordado para una etapa, un conjunto de etapas o el sistema completo. **Limitar el WIP evita tener demasiadas tareas empezadas y pocas terminadas.** Ayuda a reducir la multitarea, las acumulaciones y las esperas, y hace visibles los cuellos de botella.

Por ejemplo, si Análisis tiene un límite de 4 que abarca “En progreso” y “Hecho”, entre ambas subcolumnas no puede haber más de cuatro ítems. Terminar el análisis de uno no libera ese lugar si todavía permanece esperando que Desarrollo lo tome.

3. Gestionar el flujo

Reside en supervisar la fluidez del trabajo procurando acortar lapsos muertos, destrabar detenciones y dotar de predecibilidad a las fechas de entrega.

El eje medular apunta a **completar unidades funcionales**; de ahí que saturar de ocupación a la nómina no asegure rendimiento, pudiendo generar colas de espera indeseadas.

4. Hacer explícitas las políticas

Las políticas pueden establecer qué condiciones debe cumplir un trabajo para ingresar, cuándo puede pasar a otra etapa, qué significa estar terminado, cómo se atienden urgencias y cuáles son los límites WIP. Por ejemplo: "Una tarjeta pasa a Pruebas cuando sus cambios están integrados y superaron las verificaciones acordadas". Si Kanban se combina con Scrum, estas políticas deben respetar la DoD. Todas las políticas deben ser acordadas entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero. Las políticas deben estar expuestas en un lugar destacado.

5. Implementar circuitos de retroalimentación

Exige monitorear de manera regular el desempeño del servicio, confrontando resultados reales frente a las expectativas del cliente para articular ajustes. Estas instancias se efectivizan mediante encuentros focalizados en el estado diario, reabastecimiento, coordinación de entregas o balance general, estructurando las denominadas **cadencias de Kanban**.

6. Mejorar colaborativamente y evolucionar experimentalmente

El equipo identifica oportunidades de mejora y prueba pequeños cambios a partir de una hipótesis. Luego analiza los resultados para decidir si mantener, ajustar o descartar el cambio.

Administración de colas y sistema pull

Las **colas** contienen trabajos que esperan ser atendidos. Para evitar acumulaciones y demoras, se controla cuántos hay, cuánto tiempo llevan esperando y en qué orden se tomarán.

Kanban utiliza un **sistema pull**: cada etapa toma nuevo trabajo cuando tiene capacidad disponible, respetando los límites WIP y las políticas acordadas. En un sistema **push**, el trabajo se envía a la siguiente etapa sin esperar a que tenga capacidad.

Por ejemplo, cuando Análisis termina una tarjeta, la deja disponible para que Desarrollo la tome cuando tenga lugar. Si Pruebas alcanza su límite WIP, se busca terminar o desbloquear las tarjetas existentes antes de incorporar otras.

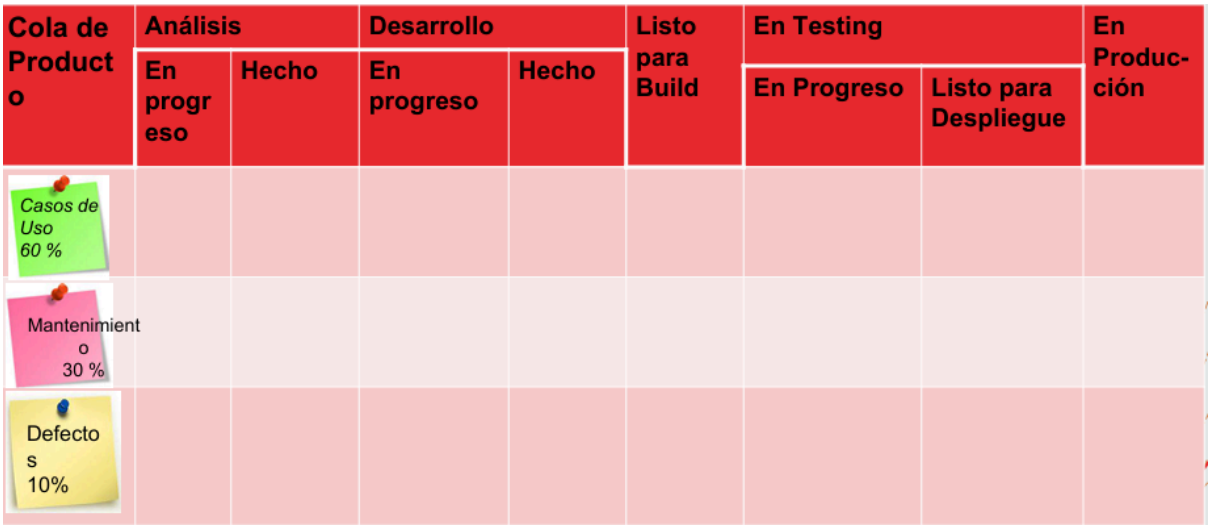
Pull indica cuándo tomar trabajo; FIFO indica cuál tomar primero. Pueden utilizarse juntos, según las políticas del equipo. FIFO es una posible política de selección no una regla obligatoria.

¿Cómo aplicar Kanban?

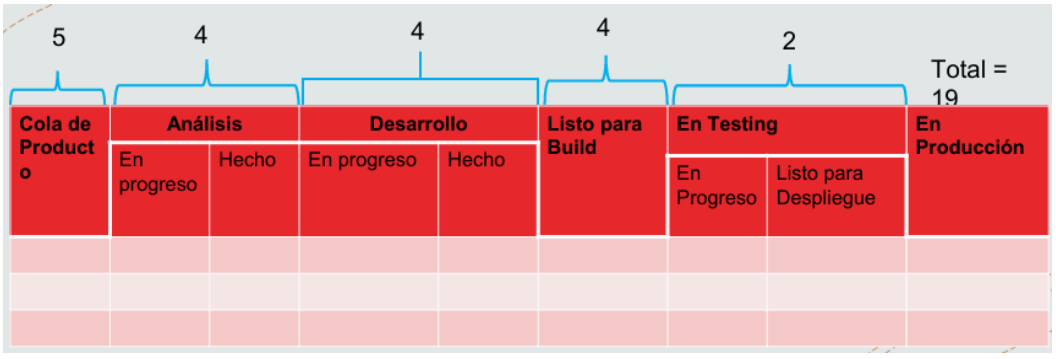
Modelar el proceso. Trazar las fases reales que atraviesan los requerimientos fijando los hitos inicial y final (ej. análisis, desarrollo, pruebas, despliegue), diferenciando labores activas e instancias de espera.



Definir las unidades y los tipos de trabajo. Precisar la naturaleza de las **tarjetas** identificando tipologías de demanda como historias de usuario, fallos, mantenimiento preventivo o cambios, balanceando la capacidad disponible.



Establecer límites de WIP. Pactar techos máximos de tareas simultáneas por etapa y ajustarlos progresivamente según las métricas operativas.



Definir políticas de calidad y funcionamiento. Explicitar pautas de ingreso, transición y finalización, y resolución de bloqueos.

Gestionar cuellos de botella. Ubicar los puntos de estancamiento y accionar sobre sus causas primarias, evaluando el impacto de cada ajuste sobre la globalidad del proceso.

Definir clases de servicio y cadencias. Formular esquemas de atención diferenciados según urgencias y estructurar la frecuencia de revisión y reabastecimiento adaptándola al entorno.

Clases de servicio

Las clases de servicio establecen cómo priorizar y atender los trabajos según el impacto que tendría retrasarlos.

Expreso o urgente. Aplica a tareas cuyo aplazamiento ocasiona daños críticos e inmediatos, como caídas del entorno productivo. Goza de máxima prioridad e interrumpe actividades en curso, por lo que su uso requiere estricta dosificación.

Fecha fija. Reagrupa requerimientos subordinados a un límite temporal estricto, vencido el cual las penalizaciones crecen drásticamente. Exige planificación anticipada.

Estándar. Abarca los requerimientos habituales cuyo diferimiento posterga beneficios sin causar emergencias, procesándose mediante reglas ordinarias.

Intangible. Comprende tareas de valor evidente pero con bajo impacto de demora inmediato, cuyo costo escala a largo plazo (e.g. refactorización de infraestructura). Conviene reservar capacidad para evitar su postergación crónica.

Cadencias

Las cadencias son reuniones que se realizan con cierta regularidad para coordinar el trabajo, revisar cómo avanza y acordar mejoras.

- **Team Kanban Meeting:** Revisar el estado y el flujo del trabajo, detectar bloqueos y decidir cómo avanzar y qué tomar si hay capacidad disponible (diaria).
- **Team Retrospective:** Reflexionar sobre cómo trabaja el equipo y acordar mejoras en su forma de gestionar el trabajo (quincenal o mensual).

- **Team Replenishment Meeting —Reposición—:** Seleccionar qué trabajos pendientes se incorporarán a continuación, según las prioridades y la capacidad disponible (semanal o a demanda).

Relación Kanban-Scrum

Scrum y Kanban son compatibles: ambos favorecen la **entrega de valor, la transparencia, la retroalimentación y la mejora continua**. Scrum organiza el trabajo mediante Sprints, objetivos, responsabilidades, eventos y artefactos. Kanban se concentra en **gestionar y mejorar el flujo**, equilibrando la demanda con la capacidad disponible mediante visualización, límites WIP y políticas explícitas, sin exigir iteraciones de duración fija.

En Scrum, un tablero puede representar el **Sprint Backlog**. En Kanban, representa el **flujo del servicio**, con sus etapas, trabajos y restricciones. Ambos pueden utilizar un sistema **pull**, tomando trabajo según la capacidad disponible.

Pueden combinarse conservando Scrum e incorporando prácticas Kanban. Por ejemplo, durante un Sprint se puede limitar a dos la cantidad de tarjetas en Pruebas, hacer visibles los bloqueos y medir tiempos de ciclo para mejorar el flujo. **Los límites WIP complementan Scrum, pero no son una exigencia del framework.**

Las cadencias pueden aprovechar reuniones existentes: la Kanban Meeting puede integrarse con el Daily Scrum, conservando su propósito, y la Retrospectiva puede incluir mejoras del flujo. La reposición se relaciona con la selección de trabajo del Sprint Planning, aunque este también define el objetivo y el plan del Sprint.

Métricas de Kanban

Lead Time (Tiempo de entrega)

Comprende el intervalo temporal total transcurrido desde que la demanda es formulada por el usuario hasta la recepción efectiva del producto finalizado, abarcando las latencias intermedias.

Se diferencia asimismo el **tiempo de entrega del sistema**, formalizado desde la asunción explícita del compromiso operativo hasta su conclusión; de allí la necesidad de explicitar los hitos delimitadores del cálculo.

Cycle Time (Tiempo de ciclo)

Contabiliza la duración requerida desde la iniciación formal del desarrollo en el circuito hasta que se considera terminada, ofreciendo un indicador de desempeño interno.

Incorpora las demoras y bloqueos suscitados; no equivale al trabajo neto volcado. Una tarea puede requerir dos jornadas de desarrollo activo pero registrar seis días de ciclo al quedar estancada en verificación.

Touch Time (Tiempo de trabajo activo)

Representa la fracción temporal exacta durante la cual se volcaron esfuerzos efectivos de elaboración, aislando el trabajo real frente a lapsos ociosos o retenciones.

Touch time <= Cycle time <= Lead Time

Eficiencia del ciclo del proceso

Proporción porcentual que mide la dedicación efectiva sobre la totalidad del tiempo transcurrido:

Eficiencia = (Touch Time / Tiempo Total Transcurrido)

Dichas variables deben homogeneizarse en la misma unidad temporal. Por caso, si un requerimiento absorbió dos días de esfuerzo activo en un período global de diez días, exhibirá una eficiencia del **0,2**, permitiendo focalizar la supresión de tiempos muertos.

Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Métricas en los 3 enfoques

Una **métrica** es una medida cuantitativa que permite describir y evaluar alguna característica de un **producto, proceso o proyecto**. Se utiliza para supervisar el trabajo, realizar estimaciones y pronósticos, evaluar resultados y detectar oportunidades de mejora. Debe definir claramente **qué se mide, cómo se calcula y en qué unidad**, e interpretarse según su contexto.

Medir tiene un **costo de planificación, recolección y análisis**. Por eso, deben elegirse métricas útiles para tomar decisiones, con una precisión acorde a la necesidad. **Se utilizan para mejorar el**

trabajo, evitando emplearlas para premiar o castigar personas, porque eso puede incentivar la manipulación de los resultados.

Cuando una característica no puede medirse directamente, pueden utilizarse **indicadores indirectos**, comprobando que representen lo que se desea conocer. Por ejemplo, la cantidad de clics puede indicar el uso de una funcionalidad, pero no demuestra por sí sola su usabilidad ni su valor.

- **Métricas de proceso:** evalúan cómo se trabaja en la organización para identificar problemas y mejorar los procesos. Reúnen información de distintos proyectos. Ejemplos: tiempos promedio, retrabajo y defectos que llegan a los usuarios.
- **Métricas de proyecto:** permiten seguir un proyecto y comparar sus resultados con lo planificado para ajustar recursos, costos y plazos. Ejemplos: esfuerzo realizado, costos reales frente a estimados y fechas de entrega.
- **Métricas de producto:** evalúan las características y la calidad del software construido para comprobar si satisface los requisitos. Ejemplos: cantidad de defectos, tiempo de respuesta y cobertura de pruebas.

Métricas en el enfoque tradicional - Métricas básicas de un proyecto de SW

Se utilizan métricas de **proceso, proyecto y producto**, con énfasis en comparar lo planificado con los resultados reales para controlar desvíos y tomar decisiones. Durante la planificación se define qué medir, cómo calcularlo, quién será responsable y cómo se analizarán los resultados.

Las métricas seleccionadas dependen del rol y de la información necesaria: por ejemplo, los testers analizan defectos y esfuerzo de pruebas; el líder del proyecto sigue costos, esfuerzo y cumplimiento de plazos. Las **métricas básicas de un proyecto de software** son:

- **Tamaño del producto:** mide cuánto software se construye, por ejemplo mediante puntos de función o líneas de código. Se relaciona con el alcance.
- **Esfuerzo:** mide el trabajo humano necesario, generalmente en horas-persona.
- **Calendario:** mide el tiempo transcurrido y permite comparar las fechas previstas con las reales.
- **Defectos:** mide los problemas detectados en el producto, considerando cantidad, severidad y otros atributos.

Estas mediciones permiten analizar el equilibrio entre **alcance, tiempo y costo**, que conforman la triple restricción del proyecto. El esfuerzo influye en el costo, mientras que el calendario refleja su duración.

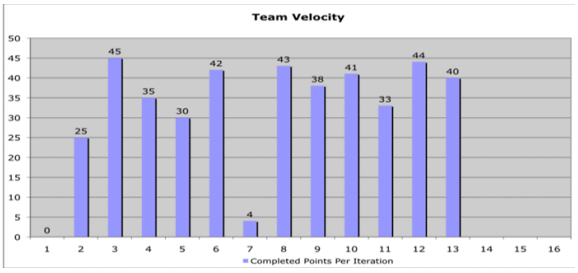
Métricas en el enfoque Lean (KANBAN)

Se orientan principalmente a **agilizar el flujo operativo y mitigar improductividades**, especialmente los tiempos muertos:

- **Lead Time:** plazo transcurrido desde la demanda inicial hasta la entrega efectiva.
- **Cycle Time:** lapso computado desde el inicio del trabajo hasta su finalización, integrando las esperas del proceso.
- **Touch Time:** duración en que la tarea recibió atención y desarrollo activo.
- **Eficiencia del proceso:** proporción del tiempo dedicado a la labor efectiva: **Touch Time / Tiempo Transcurrido × 100**.

Métricas en el enfoque Agile

Las métricas sirven para un equipo y no son extrapolables por lo que dice el agilismo a otros proyectos o equipos. En ágil hay un principio que habla específicamente de métricas (la mejor métrica de progreso es el software funcionando). Este principio apunta a que vamos a medir producto (no proceso ni proyecto). Esto es una reacción a proyectos tradicionales que tenían todas las métricas posibles cubriendo los tres enfoques, pero no teniendo avances sobre el producto (se perdía mucho tiempo y no se progresaba). Solo se debe medir lo que sea necesario y nada más, es decir, lo que agregue valor para el cliente.



- **Velocidad:** volumen de trabajo completado por Sprint y aprobado por el PO, habitualmente ponderado en puntos de historia. No se cuentan US parcialmente terminadas. Se considera metrica de producto.
- **Capacidad:** Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint. Se estima al principio de un sprint. Se puede estimar en horas ideales o en Story Points.
- **Running Tested Features (RTF):** cantidad de funcionalidades correctamente integradas que superaron las instancias de prueba.

Capacidad y velocidad distan de ser equivalentes: la primera refleja el potencial de trabajo disponible; la segunda registra las entregas efectivamente concluidas. Aunque los esquemas pueden compartir métricas, difieren en su finalidad y en el foco del análisis.

Unidad 3: Gestión de software como producto

Prácticas continuas

Las metodologías continuas permiten **incorporar, validar y liberar modificaciones pequeñas de manera recurrente** a lo largo del ciclo del producto. Su propósito radica en identificar inconsistencias de forma temprana, mitigar fricciones en la integración y acelerar la retroalimentación. Estas disciplinas se articulan mediante repositorios centralizados, automatización de procesos y ejecuciones de prueba durante el desarrollo.

Integración continua — Continuous Integration (CI)

Implica consolidar con frecuencia las contribuciones de los desarrolladores sobre un tronco común. Cada incorporación dispara la construcción automatizada de la solución y la corrida de pruebas para advertir divergencias o fallas a tiempo. Si se produce una falla, la prioridad máxima consiste en resolverla para asegurar una línea base estable. **Integrar no equivale a un despliegue productivo.**

Entrega continua — Continuous Delivery

Asegurar que el software permanezca **disponible para su puesta en producción en el momento en que el negocio lo requiera**. Extiende la integración continua incorporando la infraestructura de verificación necesaria para empaquetar una release (suma la automatización de las pruebas de aceptación). La decisión final de liberar el artefacto puede requerir una autorización deliberada.

Despliegue continuo — Continuous Deployment

Aquellas modificaciones que superan con éxito los filtros de validación se **publican automáticamente en el entorno productivo**, omitiendo aprobaciones manuales intermedias. El rasgo distintivo frente a la entrega continua estriba precisamente en la automatización completa de la puesta en producción.

Pipeline: secuencia automatizada que obtiene el código, construye el software, ejecuta controles y pruebas y genera un artefacto identificado. Puede incorporar preparación y despliegue según se utilice entrega o despliegue continuo. Sus etapas dependen del producto y sus riesgos.

Estrategias de despliegue

Canary: habilita la versión renovada inicialmente a una fracción acotada del público o del flujo de tráfico. Se monitorea la respuesta del sistema y se escala progresivamente, conteniendo el rango de impacto ante posibles contingencias.

Blue-Green: mantiene dos entornos espejados: mientras uno atiende el tráfico activo, el otro recibe la actualización. Una vez validada la versión, se conmuta el flujo operativo hacia el ambiente actualizado. Esta infraestructura simplifica eventuales reversiones, en la medida en que la compatibilidad de datos lo permita.

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Conceptos generales sobre calidad

¿Qué es la calidad del software?

La calidad del software es **el grado en que sus características le permiten cumplir los requisitos y satisfacer las necesidades de quienes lo utilizan o se ven afectados por él**. Incluye tanto las funciones que ofrece como la forma en que funcionan. Estas necesidades pueden ser:

- **Explícitas:** se expresan y documentan, cómo permitir registrar una reserva.
- **Implícitas:** se esperan aunque no se hayan detallado, como conservar los datos y proteger la información.

Por eso, cumplir las funcionalidades solicitadas es necesario, pero también deben considerarse características como confiabilidad, seguridad, rendimiento, usabilidad y mantenibilidad. Un sistema puede realizar correctamente sus cálculos y tener problemas de calidad si resulta demasiado lento o difícil de utilizar.

Calidad segun el contexto y los interesados - La calidad es subjetiva

La calidad debe evaluarse considerando **para quién se construye el producto y en qué condiciones se utilizará**. Las necesidades y expectativas varían entre personas y pueden cambiar con el tiempo. Un software de calidad debe atender las expectativas del cliente y del usuario, las necesidades de la gerencia y las del equipo de desarrollo y mantenimiento, además de otros interesados.

Principios para trabajar con calidad

- **Construir la calidad desde el inicio:** debe incorporarse en todas las actividades del desarrollo; no puede agregarse únicamente al final.
- **Responsabilidad compartida:** lograr calidad requiere el compromiso de todos, no solo del grupo de calidad o de los testers.
- **Capacitar a las personas:** necesitan conocimientos y habilidades para aplicar buenas prácticas.
- **Contar con apoyo de la dirección y liderar con el ejemplo:** los responsables deben respaldar las actividades de calidad y aplicar las prácticas que promueven. También pueden iniciarse mejoras desde el propio equipo.
- **Medir para mejorar:** utilizar información que permita conocer los resultados, detectar problemas y evaluar las mejoras.
- **Comenzar por lo básico:** aplicar prácticas simples y útiles, evitando complejidad innecesaria.
- **Planificar el aseguramiento:** definir las actividades, los responsables, los recursos y los criterios de evaluación.
- **No confundir más pruebas con más calidad:** las pruebas detectan problemas, pero la mejora requiere corregirlos y prevenir sus causas.
- **Adaptarse al negocio:** el esfuerzo de calidad debe ser razonable y proporcional a las necesidades y los riesgos del producto.

Perspectivas de la calidad - Calidad para quien?

Pressman presenta cinco perspectivas que ayudan a comprender por qué diferentes interesados pueden valorar de distinta manera un mismo producto:

Perspectiva	Cómo entiende la calidad
Trascendental	Como una excelencia que se reconoce, aunque resulte difícil definirla.
Del usuario	Como la capacidad de satisfacer sus necesidades y objetivos.
De manufactura o conformidad	Como el cumplimiento de las especificaciones y estándares establecidos.
Del producto	Como las características y propiedades que posee el producto.
Basada en el valor	Como la relación entre los beneficios obtenidos y el costo que se está dispuesto a pagar.

Estas perspectivas se complementan. Para evaluar la calidad hay que identificar **para quién debe tener calidad el producto y qué atributos son relevantes en ese contexto**.

Calidad necesaria, programada y realizada

- **Calidad necesaria:** lo que realmente necesitan los interesados.
- **Calidad programada:** lo que se planifica y especifica alcanzar.
- **Calidad realizada:** lo que efectivamente posee el producto construido.

La gestión de calidad busca que **las tres coincidan**: planificar lo necesario y construir lo planificado. Las diferencias pueden generar **insatisfacción**, cuando no se cubren necesidades, o **desperdicio**, cuando se invierte en características que no se necesitan.

Importancia de trabajar para y con calidad

Trabajar **para la calidad** implica establecer objetivos y criterios para obtener un producto adecuado. Trabajar **con calidad** implica incorporar prácticas que permitan alcanzarlos durante todo el desarrollo. La calidad debe construirse desde los requisitos y el diseño. Las pruebas ayudan a descubrir problemas, pero encontrar defectos al final obliga a corregir trabajo ya realizado y puede afectar varias partes del sistema. Por eso, la prevención y la detección temprana son fundamentales.

Trabajar con calidad busca satisfacer las necesidades y expectativas del cliente, evitando diferencias entre lo que necesita y lo que efectivamente se entrega. La prevención de defectos, la reducción del retrabajo y la mejora de costos y tiempos contribuyen a ese objetivo.

Ventajas

- **Menos retrabajo:** se evitan o descubren problemas antes de que se propaguen.
- **Menores costos de mantenimiento y soporte:** disminuyen las correcciones posteriores a la entrega.
- **Mayor satisfacción y confianza del cliente:** el producto responde mejor a sus necesidades.
- **Mayor previsibilidad:** los controles aportan información para gestionar riesgos y compromisos.
- **Mayor facilidad para evolucionar el producto:** un diseño y un código comprensibles facilitan los cambios.
- **Menor exposición a fallas:** se reducen riesgos para la operación y los usuarios.

Costos y dificultades

Trabajar con calidad requiere inversión en capacitación, planificación, revisiones, pruebas, herramientas y seguimiento. También puede generar resistencia al cambio o parecer más lento inicialmente.

Si las prácticas se aplican sin considerar el contexto, pueden producir documentación excesiva, controles redundantes y burocracia. Por eso, deben elegirse según los riesgos y necesidades del negocio. El objetivo es que el esfuerzo de calidad resulte útil y proporcional.

Costos de la calidad

Tipo de costo	Ejemplo
Prevención	Capacitación, planificación de calidad y mejora de métodos.
Evaluación	Revisiones, auditorías, mediciones y pruebas.
Fallas internas	Corrección y repetición de trabajo antes de entregar el producto.
Fallas externas	Correcciones en producción, atención de reclamos y pérdida de confianza o clientes.

La decisión debe considerar tanto el costo de trabajar con calidad como **el costo de la falta de calidad**.

Calidad de producto

Definición

La calidad de producto es **el grado en que las características del software y sus productos asociados satisfacen los requisitos y las necesidades de sus interesados**. Se evalúan resultados como requisitos, diseños, código, documentación y software funcionando. Los modelos de calidad permiten identificar características relevantes y establecer criterios para evaluarlas.

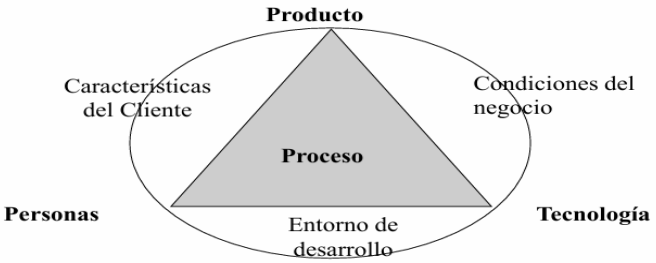
No es viable certificar calidad de producto, cada uno tiene sus propias caracteristicas, no se puede establecer un estandar de comparacion

Calidad de proceso y proceso de desarrollo

Calidad del proceso y su relación con el producto

La **calidad de proceso** es el grado en que las actividades y prácticas utilizadas para desarrollar y mantener software son adecuadas y eficaces para obtener productos que satisfagan los requisitos y las necesidades de sus usuarios.

Se insiste en la calidad del proceso porque es uno de los principales factores sobre los que la organización puede actuar de manera directa y sistemática para mejorar la calidad del producto. Sin embargo, un proceso adecuado no garantiza por sí solo un producto de calidad.



La calidad del producto depende de distintos factores: proceso, personas, tecnología, características del cliente, condiciones del negocio y entorno de desarrollo. Mejorar el proceso permite organizar cómo se utilizan esos recursos y cómo se controla el trabajo.

Para lograr un proceso de calidad debemos definirlo primero. Definir un proceso implica establecer qué actividades se realizan, quiénes son responsables, cuándo se ejecutan y qué métodos, herramientas y productos de trabajo se utilizan. El proceso debe comunicarse al equipo y adaptarse al contexto de cada proyecto, conservando los controles necesarios. Además de las actividades técnicas, incluye disciplinas transversales como planificación y seguimiento, gestión de configuración y aseguramiento de calidad.

Gestión y aseguramiento de la calidad

Gestión, aseguramiento, planificación y control

La **gestión o administración de la calidad del software** busca que el producto alcance los niveles de calidad requeridos. Para ello, establece procesos y estándares adecuados, planifica cómo aplicarlos en cada proyecto y controla su cumplimiento. También promueve una **cultura de calidad**, donde todos se responsabilizan por la calidad de su trabajo. Incluye:

Actividad	Propósito

Aseguramiento —QA/SQA—	Establecer y evaluar procesos, estándares y actividades para proporcionar confianza en que se cumplirán los requisitos de calidad.
Planificación	Determinar qué calidad se busca, qué estándares se aplicarán y cómo se evaluará en el proyecto o producto.
Control —QC—	Comprobar el cumplimiento de los criterios, estándares y procedimientos establecidos y detectar problemas mediante la evaluación de procesos y resultados. Se puede realizar mediante revisiones de calidad o evaluaciones automáticas.

Por ejemplo, establecer un procedimiento de revisión corresponde al aseguramiento; decidir qué componentes se revisarán y cuándo, a la planificación; examinarlos y registrar defectos, al control. Las actividades se relacionan y pueden utilizar técnicas comunes.

Grupo de aseguramiento de calidad — GAC

El GAC proporciona una **evaluación objetiva** y ayuda a que el equipo alcance sus objetivos de calidad. Debe contar con suficiente independencia para informar problemas aunque exista presión por entregar. La filmina recomienda que no reporte al gerente del proyecto evaluado, que esté próximo a la dirección y que reporte a alguien interesado en la calidad.

Esta independencia **no reemplaza la responsabilidad del equipo de desarrollo** sobre la calidad de su trabajo.

Actividades del aseguramiento de calidad

El aseguramiento acompaña el ciclo de vida completo mediante actividades como:

- **Preparar el plan de aseguramiento:** definir evaluaciones, estándares, responsables y tratamiento de problemas.
- **Participar en la definición y adaptación del proceso:** comprobar su adecuación a las políticas organizacionales y al proyecto.
- **Evaluar la planificación y el seguimiento del proyecto.**

- **Evaluar las actividades de desarrollo:** requisitos, diseño, programación, integración y pruebas.
- **Evaluar productos de trabajo:** documentos, modelos, código y otros resultados.
- **Proporcionar o promover métodos, técnicas y herramientas adecuados** para las evaluaciones.
- **Registrar y seguir desviaciones:** asignar responsables y controlar su resolución.
- **Informar a la dirección:** dar visibilidad y elevar problemas que requieren decisiones.
- **Analizar métricas y causas:** identificar problemas recurrentes y oportunidades de mejora.

4.4. Planificación y Plan de Aseguramiento de Calidad de Software — PACS

La planificación determina **qué atributos de calidad son importantes y cómo se comprobarán**. Selecciona y adapta estándares organizacionales y establece otros cuando sea necesario. El PACS concreta las actividades de aseguramiento e incluye:

- Objetivos y alcance.
- Estándares y procedimientos aplicables.
- Actividades, productos y criterios de evaluación.
- Responsables, recursos y fechas.
- Revisiones y auditorías previstas.
- Registros e informes.
- Tratamiento de desviaciones y verificación de su cierre.

El PACS es uno de los planes de soporte del proyecto y se coordina con el plan de proyecto. Define las actividades de aseguramiento que acompañarán el desarrollo del producto.

4.5. Estándares de producto y de proceso

Los estándares proporcionan criterios comunes para realizar y evaluar el trabajo. Pueden ser internacionales, nacionales, organizacionales o específicos de un proyecto.

- **Estándares de producto:** establecen características que deben cumplir los productos de trabajo. Por ejemplo, convenciones de programación o contenido de documentos.
- **Estándares de proceso:** establecen cómo deben ejecutarse las actividades. Por ejemplo, cómo realizar revisiones o gestionar cambios.

Se debe comprobar su cumplimiento, pero también su pertinencia. **No corresponde conservar una práctica inadecuada solamente porque esté estandarizada.**

Control y revisiones de calidad

El control utiliza **revisiones, evaluaciones automatizadas y mediciones** para comprobar los criterios establecidos y detectar problemas. En una revisión, un grupo examina un proceso o producto y su documentación. Según el objetivo, puede buscar:

- **Remover defectos:** inspecciones del producto.
- **Evaluar el progreso:** revisión del estado del proyecto y sus resultados.
- **Comprobar estándares y calidad:** revisión de productos y prácticas.

Las revisiones técnicas, auditorías y pruebas se desarrollan en sus apartados específicos.

Evaluación y mejora del proceso

La mejora del proceso consiste en evaluar cómo se trabaja y los resultados obtenidos, identificar causas de problemas e implementar cambios para mejorar la calidad, los costos o los tiempos. Requiere compromiso organizacional, recursos y seguimiento.

Modelos de mejora y normas de gestión de calidad

5.1. Propósito y alcance

Los **modelos de mejora y las normas de calidad** proporcionan criterios y buenas prácticas para definir, evaluar y mejorar la forma de trabajar y sus resultados. **SPICE y CMMI** orientan la evaluación y mejora de los procesos; **ISO 9001** establece requisitos para un sistema de gestión de la calidad; y la familia **ISO/IEC 25000 —SQuaRE—** orienta los requisitos, la medición y la evaluación de la calidad del producto. Por eso, se complementan, pero no evalúan exactamente lo mismo.

5.2. SPICE no tiene areas de proceso, es solo para los que tenemos

SPICE es un enfoque de evaluación y mejora de la capacidad de los procesos por separado, históricamente asociado con **ISO/IEC 15504**. Permite conocer la situación actual, identificar debilidades y establecer objetivos de mejora. Distingue dos dimensiones:

- **Procesos:** qué procesos se evalúan y qué resultados deben producir.
- **Capacidad:** qué tan adecuadamente se ejecutan, gestionan y mejoran.

Su evaluación se centra en procesos específicos; no debe confundirse automáticamente con una clasificación de madurez de toda la organización. La capacidad se expresa en 6 niveles:

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predecible	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizando	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

5.3. Modelo IDEAL

IDEAL es un modelo desarrollado por el SEI que guía la **iniciación, planificación e implementación de mejoras en los procesos de una organización**. Las iniciativas de mejora se gestionan como **proyectos**, con objetivos, responsables, recursos, plazos y apoyo de la dirección —*sponsor*—.

Es **cíclico**: se analiza la situación actual, se implementan mejoras y se aprende de los resultados para orientar el siguiente ciclo. Sus cinco fases son:

1. **Iniciar:** reconocer la necesidad de mejorar, definir el contexto y los objetivos generales, conseguir apoyo y organizar los recursos.
2. **Diagnosticar:** evaluar los procesos actuales, identificar fortalezas y debilidades y determinar la **brecha entre la situación actual y la deseada**, utilizando referencias como CMMI o SPICE.
3. **Establecer:** fijar prioridades y elaborar un plan de acción con actividades, responsables, recursos y plazos.
4. **Actuar:** desarrollar e implementar las mejoras, probarlas mediante **pilotos**, ajustarlas y extenderlas a la organización.
5. **Aprender:** evaluar los resultados y registrar lecciones aprendidas para mejorar los procesos y la forma de conducir futuras iniciativas.

El resultado puede ser un **proceso organizacional definido o mejorado**, documentado y acompañado de guías para adaptarlo a los proyectos. Se busca contribuir a mejores productos y resultados, como reducir defectos, costos o tiempos. **IDEAL organiza cómo conducir la mejora; CMMI y SPICE proporcionan referencias para evaluar los procesos y orientar qué mejorar.**

Relación con la evaluación formal: si se busca obtener un nivel de capacidad o madurez, se prepara una evaluación con el método correspondiente. Antes de la visita se define el alcance y se revisan documentación y evidencias; durante la evaluación se examinan proyectos seleccionados, registros, productos de trabajo y entrevistas. La preparación permite identificar brechas, pero **no garantiza alcanzar el nivel buscado**. Esta evaluación es una actividad diferenciada, no una sexta fase de IDEAL.

Modelo de calidad - CMMI (ver siglas)

Modelos de referencia para evaluar procesos

Reúnen buenas prácticas para evaluar y mejorar los procesos, comparando su ejecución y evidencias con el modelo. Permiten identificar fortalezas y debilidades y determinar la **madurez organizacional o la capacidad de procesos específicos**. A diferencia de ellos, una **auditoría de proyecto** se centra en comprobar el cumplimiento de los procesos, estándares y procedimientos aplicables.

Los modelos de calidad y mejora de procesos proporcionan **objetivos y buenas prácticas para definir o mejorar los procesos de una organización**. Indican qué debe lograrse, pero no imponen una única forma de hacerlo: la organización decide cómo implementarlos y documenta sus procesos para que sean accesibles a los involucrados.

Estos procesos deben incluir **guías de adaptación** que permitan ajustarlos a las características de cada proyecto. Al planificarlo, se establece qué proceso se utilizará y se documenta en el plan de proyecto o en documentos referenciados. Luego, las auditorías comprueban que el trabajo realizado sea consistente con los procesos, estándares y compromisos aplicables.

Definición y evaluación

CMMI —Capability Maturity Model Integration— es un modelo de referencia que reúne buenas prácticas para mejorar y evaluar procesos organizacionales. Evoluciona a partir de modelos anteriores como SW-CMM. Orienta sobre los objetivos que deben alcanzarse; la organización determina cómo implementarlos en su contexto particular.

La evaluación contrasta prácticas y evidencias con el modelo. No equivale a certificar que un producto particular está libre de defectos. En el marco de CMMI v1.3 se utilizaban métodos de evaluación como **SCAMPI**.

Antes de una evaluación formal se analiza la diferencia entre las prácticas actuales y lo requerido por el modelo, identificando brechas y acciones de mejora. La evaluación recopila evidencias de proyectos y procesos dentro de un alcance y una muestra definidos para determinar el cumplimiento.

Constelaciones

Reúne los componentes de CMMI adaptados a un ambito de aplicacion.

- **Desarrollo - CMMI-DEV:** provee la guía para medir, monitorear y administrar los procesos de desarrollos de software. Tiene dos representaciones, por etapas y continua.
- **Adquisición - CMMI-ACQ:** provee la guía para permitir seleccionar, administrar y adquirir productos y servicios a otra empresa que posee su propia forma de trabajo, delegando el control de ese proyecto para obtener el producto o servicio final.
- **Servicios - CMMI-SVC:** provee la guía para entregar servicios, externos o internos.

Estas constelaciones poseen los modelos, capacitaciones y toda la información que es necesaria para que las empresas puedan acreditar distintos niveles de madurez

Roles y grupos

Las actividades deben contar con responsables identificados. Según el tamaño y las necesidades de la organización, una función puede estar cubierta por una persona o por un grupo. Lo importante es **que las responsabilidades se cumplan**, sin exigir un departamento separado para cada función.

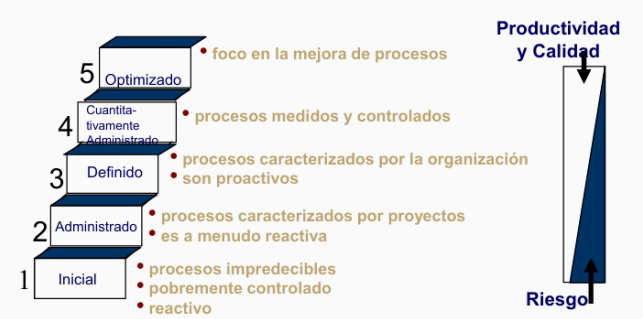
CMMI puede aplicarse mediante dos representaciones:

- **Por etapas:** evalúa la **madurez organizacional** mediante cinco niveles acumulativos, del 1 al 5. Propone un camino de mejora con áreas de proceso establecidas para cada nivel.
- **Continua:** evalúa la **capacidad de áreas de proceso específicas**, seleccionadas según las necesidades de la organización. Permite mejorarlas individualmente mediante niveles del 0 al 5.

6.4. Representación por etapas — Madurez organizacional

Evalúa la **madurez de los procesos dentro del alcance organizacional evaluado** y propone un camino de mejora mediante cinco niveles. Cada nivel reúne áreas de proceso cuyos objetivos deben cumplirse. **Los niveles son acumulativos:** alcanzar uno requiere satisfacer también los requisitos aplicables de los anteriores.

1. **Inicial:** los procesos son poco controlados y los resultados, impredecibles. El éxito depende principalmente del esfuerzo individual; son frecuentes los retrasos y sobrecostos.
2. **Administrado:** los proyectos gestionan sus requisitos de manera efectiva y planifican, ejecutan, supervisan y controlan sus procesos y productos de trabajo.
3. **Definido:** existen procesos organizacionales documentados y comprendidos con mas rigurosidad que el nivel 2, que se adaptan a cada proyecto. Se brinda capacitaciones de personal y un nivel mas avanzado de metricas.
4. **Administrado cuantitativamente:** Se caracteriza porque las organizaciones disponen de un conjunto de métricas significativas de calidad y productividad, que se usan de modo sistemático para la toma de decisiones y la gestión de riesgos. El software resultante es de alta calidad. Las medidas detalladas del rendimiento de los procesos son recogidas y analizadas estadísticamente.
5. **Optimizado:** se mejoran continuamente los procesos mediante el análisis de causas y la incorporación de mejoras e innovaciones alineadas con los objetivos del negocio.



Esta representación facilita comparar la madurez de organizaciones, considerando el alcance de sus evaluaciones. Hacemos **foco en el nivel 2.** Incluye siete áreas de proceso:

- Administración de requerimientos.
- Planificación de proyectos.
- Monitoreo y control de proyectos.
- Administración de acuerdos con proveedores.
- Medición y análisis.
- Aseguramiento de calidad de proceso y producto —PPQA—.
- Administración de configuración.

Para alcanzar el nivel de madurez 2 deben satisfacerse los objetivos específicos de todas las áreas aplicables de ese nivel y el objetivo genérico correspondiente a su institucionalización como procesos administrados. No alcanza con definir procedimientos: deben aplicarse, contar con responsables y recursos, supervisarse y evaluarse.

Administración de acuerdos con proveedores se aplica cuando existen adquisiciones comprendidas en su alcance; su exclusión debe justificarse.

Alcanzar el nivel 2 indica que existen bases para **gestionar los proyectos y obtener resultados más previsibles**, aunque no garantiza por sí solo la calidad de cada producto.

6.5. Representación continua — Capacidad de los procesos

El objetivo del uso de esta representación, es acreditar la capacidad de la organización ante cada una de las áreas de proceso de forma individual. En lugar de medir la madurez de toda la organización, mido la capacidad de un proceso en particular. Procesos de nivel cero son los que no se ejecutan (Por ejemplo: si le pregunto a una empresa si hace gestión de configuración de software y me dice que no lo hace, entonces, es nivel 0 en esa área). Apunta a mejorar áreas de proceso que uno elige. La escala utilizada en este caso es:

0. Incompleto: el proceso no se realiza o se realiza parcialmente, sin alcanzar todos sus objetivos.

1. Realizado: el proceso alcanza sus objetivos y genera los resultados esperados.

2. Administrado: se planifica, ejecuta, supervisa y controla, con responsables y recursos definidos.

3. Definido: utiliza un proceso estándar de la organización, adaptado al contexto del proyecto.

4. Administrado cuantitativamente: se controla mediante mediciones y técnicas estadísticas para lograr un desempeño predecible.

5. En optimización: se mejora continuamente mediante el análisis de causas y la incorporación de mejoras e innovaciones.

6.6. Áreas de proceso por nivel — CMMI-DEV v1.3

Un **área de proceso** es un conjunto de prácticas relacionadas que, aplicadas conjuntamente, permiten alcanzar objetivos de mejora en un aspecto del proceso. CMMI-DEV v1.3 contiene **22 áreas de proceso**, presentes en ambas representaciones. El **nivel 1 no tiene áreas asignadas** y el **nivel 2 incluye siete**:

- **REQM — Administración de requerimientos:** gestionar los requisitos y mantener su coherencia con los planes y productos de trabajo. Req identificados, con trazabilidad y control de cambios a lo largo del ciclo de vida.
- **PP — Planificación de proyectos:** establecer estimaciones, actividades, recursos y compromisos.

- **PMC — Monitoreo y control de proyectos:** comparar el avance con lo planificado y tomar acciones correctivas.
- **MA — Medición y análisis:** obtener y analizar metricas para apoyar las decisiones.
- **PPQA — Aseguramiento de calidad de proceso y producto:** evaluar objetivamente el cumplimiento de procesos y criterios, y comunicar incumplimientos.
- **CM — Administración de configuración:** mantener la integridad de los productos de trabajo mediante su identificación, control de cambios y seguimiento.
- **OPCIONAL - SAM — Administración de acuerdos con proveedores:** Definir todo lo anterior pero para proveedores. Todas las practicas que no tiene el proveedor debemos darselas nosotros. Si nadie interviene de forma externa no debo definir practicas, pero si por ej el testing lo hace un tercero si.

Las áreas también se agrupan en cuatro categorías:

- **Administración de proyectos:** planificación, seguimiento, riesgos y coordinación.
- **Administración de procesos:** definición, despliegue, medición y mejora de los procesos organizacionales.
- **Ingeniería:** requerimientos, solución técnica, integración, verificación y validación.
- **Soporte:** actividades que sostienen el trabajo, como PPQA, configuración y medición.

Nivel	Categoría			
	Administración de Proyectos	Soporte	Administración de Procesos	Ingeniería
5 Optimizado		▪ Análisis y Causal y Resolución (CAR)	▪ Administración de Performance Organizacional (OID)	
4 Cuantitativamente Administrado	▪ Administración Cuantitativa del Proyecto (QPM)		▪ Performance del Proceso Organizacional (OPP)	
3 Definido	▪ Administración de Riesgos (RSKM) ▪Administración Integrada de Proyectos (IPM)	▪Análisis y Resolución de Decisión (DAR)	▪ Definición del Proceso Organizacional (OPD) ▪ Foco en el Proceso Organizacional (OPF) ▪ Capacitación Organizacional (OT)	• **Desarrollo de Requerimientos (RD)** • **Solución Técnica (TD)** • **Integración de Producto (PI)** • **Verificación (VER)** • **Validación (VAL)**
2 Administrado	▪Administración de Requerimientos (REQM) ▪Planificación de Proyectos (PP) ▪ Monitoreo y Control de Proyectos (PMC) ▪ Administración de Acuerdo con el Proveedor (SAM)	▪ Aseguramiento de calidad de Proceso y de Producto (PPQA) ▪ Administración de Configuración (CM) ▪ Medición y Análisis (MA)		
1 Inicial	**Procesos sin definir o improvisados**			

7. Relación entre CMMI y Agile

Ambos buscan mejorar los resultados del desarrollo, pero tienen **distintos énfasis**. CMMI se concentra en la capacidad y madurez de los procesos organizacionales; Agile enfatiza entregar valor y adaptarse mediante retroalimentación frecuente.

Contraste y compatibilidad

Las prácticas ágiles pueden apoyar objetivos de CMMI, como planificar, asignar responsabilidades, involucrar interesados, gestionar la configuración y mejorar continuamente.

Pueden aparecer tensiones al implementar políticas organizacionales, evaluaciones objetivas y controles que el equipo perciba como sobrecarga. **La compatibilidad depende de cómo se implementen:** usar Agile no demuestra automáticamente cumplimiento de CMMI.

Diferencias en valores esenciales

- **Prioridad:** CMMI enfatiza medir y mejorar procesos; Agile, responder al cliente y reducir la sobrecarga.
- **Comunicación:** CMMI destaca la coordinación organizacional; Agile, la comunicación directa entre personas.
- **Conocimiento:** CMMI busca conservarlo en procesos y activos organizacionales; Agile destaca el conocimiento compartido mediante la colaboración.
- **Personas:** la filmina contrapone disciplina y seguimiento de reglas con creatividad y disposición a experimentar. Son énfasis ilustrativos: ambas formas de trabajo necesitan disciplina y creatividad.

Diferencias en características

- **Mejora:** CMMI enfatiza institucionalizar mejoras en la organización; Agile, inspeccionar y adaptar el trabajo del equipo y el producto.
- **Resultados:** CMMI busca consistencia y previsibilidad; Agile destaca aprovechar oportunidades y responder a cambios.
- **Confianza del cliente:** CMMI aporta evidencia de procesos gestionados; Agile enfatiza software funcionando y participación de los interesados.
- **Organización del trabajo:** CMMI destaca estructuras y responsabilidades organizacionales; Agile, autonomía y colaboración de los equipos.

Diferencias de enfoque y foco

- **CMMI:** énfasis organizacional y estratégico, procesos, medición, estabilidad y previsibilidad.
- **Agile:** énfasis en el producto, el cliente, la adaptación, la entrega frecuente y la rapidez de respuesta.
- **CMMI indica objetivos y prácticas esperadas**, dejando su implementación al contexto. Los marcos y métodos ágiles aportan formas concretas de organizar el trabajo.

Estas diferencias no son exclusiones: Agile también mide, planifica y previene riesgos; CMMI también admite adaptación e innovación. Por eso, no conviene memorizar literalmente "Agile es solo cualitativo" o "gestiona riesgos de forma reactiva".

Similitudes

- Buscan organizaciones de alto desempeño.
- Ambos planifican y establecen reglas de trabajo.
- Se apoyan en experiencia y aprendizaje.
- Ninguno constituye una solución completa.
- Su aplicación debe adaptarse al contexto: ninguno sirve de la misma manera para cualquier proyecto.

Auditorías al software

¿Qué es una auditoría?

Una auditoría es una **evaluación sistemática, objetiva e independiente** que utiliza evidencias para determinar el cumplimiento de criterios establecidos: estándares, procedimientos, especificaciones o acuerdos. Puede examinar procesos y productos de trabajo. Permite identificar incumplimientos, reconocer buenas prácticas y dar visibilidad a la dirección sobre cómo se trabaja.

Como beneficios involucra evaluar el cumplimiento del proceso, determinar la implementación efectiva del proceso organizacional y del proyecto, y actividades de soporte, y dar visibilidad a la gerencia sobre los procesos.

La auditoría requiere objetividad e independencia respecto del trabajo auditado. La organización debe evitar conflictos de interés; una línea de reporte diferente puede contribuir a esa independencia.

Auditoría de proyecto

Evalúa objetivamente si el proyecto cumple el proceso definido y adaptado a su contexto. Contrasta evidencias del trabajo realizado con los procedimientos y documentos establecidos, verificando también la consistencia entre requisitos, diseño, código y pruebas. Se planifica y ejecuta según el **PACS**.

Auditorías de configuración

Se hacen mientras se ejecuta el proyecto y ven si el producto es íntegro.

Auditoría funcional: La auditoría funcional compara el software que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la ERS. El propósito de la auditoría funcional es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descriptos en la ERS. El responsable de QA deberá validar si la matriz de rastreabilidad está actualizada.

Auditoría física: compara el código desarrollado con su documentación de soporte para verificar que sean consistentes y que la documentación describa correctamente el software que se entregará. El **PACS** establece quién la realiza y las desviaciones encontradas deben corregirse antes de la entrega.

Auditoría al grupo de calidad

Las auditorías al grupo de calidad o externa son evaluaciones objetivas que comprueban si la propia área de calidad de la organización cumple sus responsabilidades y los procedimientos establecidos para asegurar la calidad. Se realiza un plan de calidad para evaluar a los auditores. Para ello se examinan planes, registros e informes. Son realizadas por gente externa a la organización.

Proceso de auditorías

Roles y responsabilidades

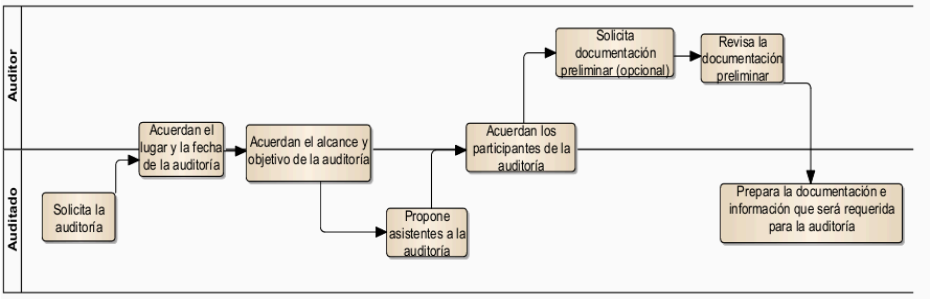
Rol	Responsabilidad principal
Responsable de SQA	Planificar auditorías, asignar recursos y asegurar el tratamiento de las no conformidades.
Auditor	Acuerda la fecha, comunicar el alcance, reunir y analizar evidencias, realiza la auditoria, informar resultados y verificar el seguimiento.
Auditado	Acuerda fecha, proporcionar evidencia, participar, responder al informe y realizar las acciones acordadas.

El auditor debe mantener objetividad y una comunicación respetuosa. El propósito es evaluar el cumplimiento y mejorar el trabajo.

Etapas -> Es el mismo sin importar la auditoria

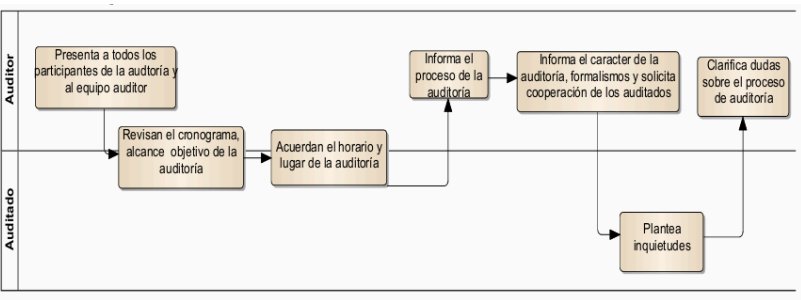
1. Preparación y planificación

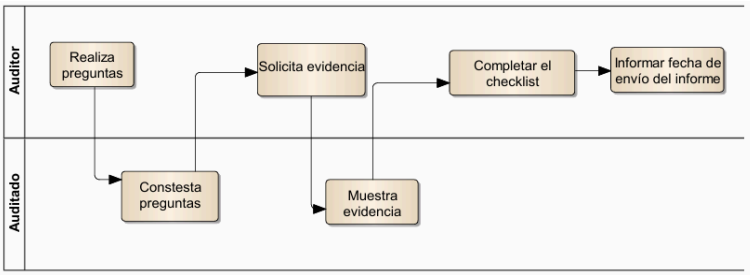
El auditado solicita la auditoría y junto al auditor acuerdan fecha, alcances, objetivos, participantes y métricas de calidad



2. Ejecución

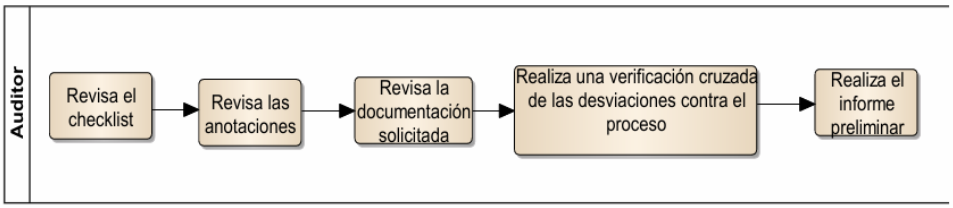
Comienza con una reunión de apertura para confirmar el alcance y la dinámica. Luego sucede la ejecución propiamente dicha: se realizan entrevistas, se examinan registros y productos y se contrastan evidencias con los criterios.

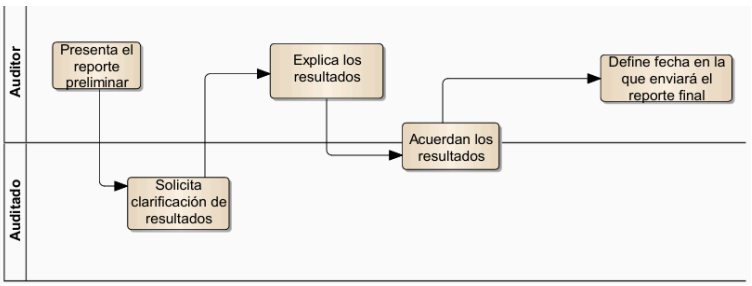


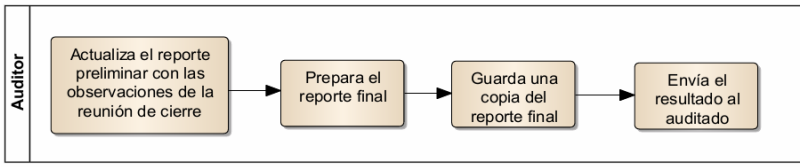


3. Análisis e informe

El auditor evalúa los hallazgos y comprueba que sus conclusiones tengan respaldo. En la reunión de cierre comunica los resultados y aclara dudas. Después emite el informe final.

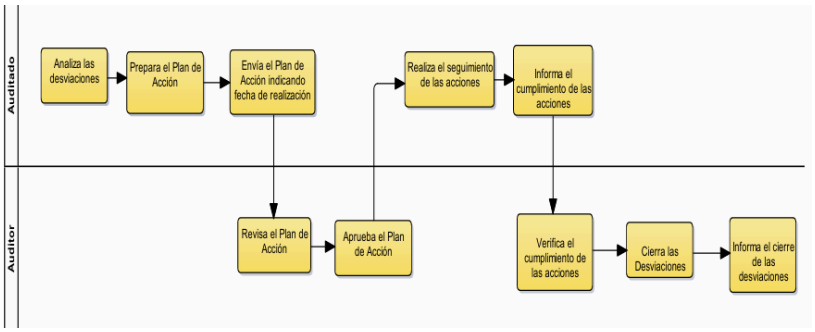






4. Seguimiento

El auditado propone acciones con responsables y fechas. Se comprueba su realización y eficacia antes de cerrar las desviaciones. Informar que algo se corrigió debe acompañarse de evidencia de la corrección.



Técnicas y herramientas

- **Checklists:** listas de preguntas que establecen un conjunto mínimo de aspectos que deben verificarse, independientemente de quién realice la auditoría. Orientan la evaluación, pero el auditor puede realizar preguntas adicionales y solicitar más evidencias. Reúne verificaciones mínimas para asegurar cobertura y favorecer evaluaciones consistentes. No reemplaza el juicio del auditor ni garantiza por sí solo su independencia.
- **Muestreo:** selección de una muestra representativa de productos, procesos o evidencias para examinarlos.
- **Revisión de registros:** análisis de documentos y evidencias del trabajo realizado para comprobar el cumplimiento de los criterios establecidos.
- **Herramientas automatizadas:** permiten recopilar y analizar información, registrar hallazgos y hacer seguimiento de las acciones acordadas.

Resultados o Hallazgos

- **Buenas prácticas:** formas de trabajo especialmente efectivas, que superan lo esperado. Por ejemplo, una herramienta que facilita el acceso, la actualización y el seguimiento del plan del proyecto.
- **Desviaciones o No conformidad:** incumplimientos del proceso definido, los estándares o los requisitos documentados. Según la clasificación utilizada en clase, pueden distinguirse:
    - **No adecuado:** la práctica no se realiza o se realiza incorrectamente.

    - **Necesita mejora:** la práctica se realiza, pero presenta deficiencias respecto de lo requerido.

Las desviaciones **requieren un plan de acción por parte del auditado**.

- **Observaciones:** situaciones que no constituyen un incumplimiento, pero podrían generar riesgos o futuras desviaciones. Se señalan para que el equipo considere mejorarlas y **no requieren un plan de acción formal**. Por ejemplo, una técnica de estimación que cumple el procedimiento, pero muestra resultados poco confiables.

Reporte de auditoría

Es el documento que registra el alcance de la auditoría, las evidencias examinadas, los hallazgos y las conclusiones. Debe informar, entre otras:

- Desviaciones que provocan que un producto no cumpla sus requisitos.
- Desviaciones respecto del proceso definido o de los requisitos documentados.

También comunica las buenas prácticas, las observaciones y las solicitudes de planes de acción.

Testing: Calidad de producto: planificación y realización de pruebas

Testing en el contexto de la calidad

La calidad del software se construye durante todo el desarrollo mediante actividades que evalúan y mejoran tanto el proceso como el producto. La **gestión de configuración SCM** proporciona una base de versiones, trazabilidad y control de cambios que permite realizar esas evaluaciones sobre productos identificados. El **testing contribuye al control de calidad del producto**, detectando problemas y aportando información sobre su comportamiento. Complementa las revisiones y auditorías, pero no garantiza la calidad por sí solo: **el aseguramiento de calidad —QA— es más amplio que el testing**.

¿Qué es testing?

El **testing de software** comprende actividades destinadas a evaluar un sistema o componente, detectar defectos y aportar información sobre su calidad y sus riesgos. Las pruebas dinámicas ejecutan el software bajo condiciones específicas y comparan los resultados obtenidos con los esperados según los requisitos.

Se considera una actividad **"destructiva"** porque busca activamente situaciones en las que el software falla: tanto cuando no hace lo requerido como cuando realiza algo no permitido. No significa dañar el

producto. El testing contribuye al **control de calidad** y forma parte del esfuerzo más amplio de aseguramiento. Se planifica y diseña desde etapas tempranas.

Las pruebas **pueden mostrar la presencia de defectos, pero no demostrar su ausencia**. Encontrarlos aporta valor, aunque una prueba que pasa también proporciona información: no encontrar defectos no significa automáticamente que el testing haya sido inútil.

Como las pruebas exhaustivas suelen ser inviables, se seleccionan casos y cobertura según los **riesgos, prioridades y recursos disponibles**. Alcanzar un 100 % en una medida de cobertura no implica haber probado todas las situaciones posibles.

Principios del testing

1. Las pruebas muestran la presencia de defectos, pero no demuestran su ausencia.
2. Las pruebas exhaustivas son inviables para sistemas no triviales.
3. Probar tempranamente reduce problemas y retrabajo.
4. Los defectos suelen concentrarse en algunas partes del sistema.
5. Repetir siempre las mismas pruebas puede dejar de revelar nuevos problemas.
6. La estrategia depende del contexto.
7. Un producto sin defectos detectados puede seguir siendo inútil si no satisface necesidades.

Mitos del testing

Es incorrecto afirmar que el testing demuestra que **no existen defectos** o que basta con comprobar que el programa hace lo esperado. Las pruebas buscan detectar problemas y aportar información sobre la calidad. Deben comprobar tanto que el software **haga lo requerido** como que **no realice acciones no permitidas**. Superar las pruebas brinda confianza sobre lo evaluado, pero **no garantiza que el software esté libre de defectos**.

Cuando el testing es suficiente?

El testing se considera suficiente cuando se cumplen los criterios de aceptación acordados y el riesgo restante se considera aceptable. No significa haber probado todo y demostrado que no existen defectos, porque las pruebas exhaustivas por principio son inviables debido al tiempo y el testing no demuestra que no existen defectos.

Error, defecto y falla

- **Error:** equivocación humana. El problema se detecta y se corrige dentro de la misma etapa en la que se originó.

- **Defecto:** problema introducido en un requisito, diseño, código u otro producto de trabajo. Es un problema que no se detectó en su etapa de origen y se trasladó a una etapa posterior.
- **Falla:** comportamiento incorrecto que se manifiesta durante la ejecución. Un defecto puede permanecer sin manifestarse hasta que se presentan determinadas condiciones. Ambos pueden o no generar fallas en el sistema.

Severidad y prioridad de los defectos

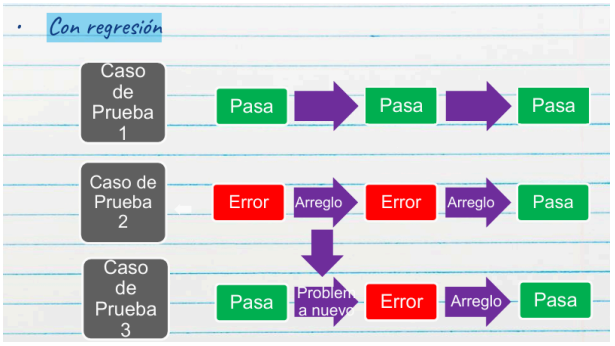
La severidad define la gravedad del defecto y es determinada por la persona que realiza el testing por lo que es de carácter técnico.

Severidad	Significado
Bloqueante	Impide utilizar el sistema o continuar la actividad evaluada.
Crítico	Impide realizar una funcionalidad importante y no existe una alternativa.
Mayor	Impide alcanzar el objetivo por el recorrido previsto, pero existe una alternativa.
Menor	Afecta parcialmente el comportamiento sin impedir la operación principal.
Cosmético	Afecta presentación, ubicación, tipografía u otros aspectos de forma.

La **prioridad** define el impacto que tiene el defecto en la funcionalidad del negocio y permite ordenar la atención de los defectos según necesidades del cliente. Son: Urgencia, Alta,Media,Baja

Severidad y prioridad no son equivalentes y varían dependiendo el negocio. Un problema de presentación puede tener alta prioridad si afecta una demostración importante; una falla grave en una función poco utilizada puede tener menor urgencia inmediata.

Ciclo de pruebas



Un ciclo de pruebas comprende la ejecución del conjunto de casos de prueba seleccionados sobre una **misma versión del software**, utilizando un entorno y datos identificados. Se registran resultados, defectos y condiciones de ejecución. Cuando las correcciones generan una nueva versión, se comprueba que los defectos fueron solucionados mediante pruebas de **confirmación** y se realizan pruebas de **regresión** para detectar efectos sobre comportamientos existentes que funcionan. El **proceso de**

pruebas comprende planificación, diseño, ejecución y evaluación; un **ciclo** es una ejecución concreta del conjunto definido.

Casos de prueba

Un caso de prueba describe qué comportamiento del software se quiere comprobar, bajo qué precondiciones, con qué datos de entrada, mediante qué acciones y qué resultado se espera. Su propósito es detectar posibles defectos y evaluar el cumplimiento de lo esperado, comparándolo con el resultado obtenido. De esta forma se componen de 3 partes: Objetivo (caracteristica que se desea probar), Datos de entrada y ambiente (precondiciones) y Comportamiento esperado

Los casos se derivan de requisitos, historias de usuario, criterios de aceptación, reglas de negocio y otras fuentes que permiten determinar el comportamiento esperado.

Diseñar una prueba consiste en determinar la situación que se evaluará, los datos, las acciones y los resultados esperados. **Ejecutarla** consiste en aplicarla a una versión del software, registrar el resultado obtenido y compararlo con el esperado. Por eso, pueden diseñarse casos antes de tener el sistema construido.

Niveles de prueba

Los niveles de prueba determinan el foco o la granularidad de la prueba que se está por ejecutar, comenzando por los componentes pequeños y luego aumentando la granularidad.

- **Unitario o componente:** Se realizan sobre un componente individual de manera independiente a los demás. Se hacen teniendo acceso al código fuente y con el apoyo del entorno de desarrollo. Estas pruebas son realizadas principalmente por los desarrolladores durante la implementación.
- **Integración:** Verifican el funcionamiento de dos o mas componentes que se relacionan con peticiones mediante sus interfaces. Pueden automatizarse y ejecutarse dentro de la integración continua CI. Se suele llevar a cabo una integración incremental comenzando desde lo general a lo especifico (top-down) o desde lo especifico a lo general (bottom-up)
- **Sistema:** evalúa el comportamiento del sistema completo o de un incremento, considerando requisitos funcionales y no funcionales. La realizan personas ajenas al desarrollo bajo un proceso sistemático donde el entorno de prueba debe ser lo más similar a producción para evitar errores.
- **Aceptación:** evalúa si el software satisface las necesidades y los criterios del negocio o del usuario. Aquí, los usuarios o representantes del negocio comprueban que la solución les permite operar como necesitan, con acompañamiento del equipo. Sus resultados contribuyen a

decidir si el producto está listo para ponerse en producción. Su foco principal no es hallar defectos sino observar si el usuario esta conforme con el producto. Comprende tanto las pruebas en el ambiente de laboratorio como de trabajo real.

Cada nivel busca problemas diferentes. Superar las pruebas unitarias no reemplaza la evaluación de las interacciones ni del producto completo.

Ambientes para construir y probar software

Son conjuntos de recursos y configuraciones (HW,SW,herramientas, BD,etc) preparados para desarrollar, probar u operar el sistema. Separar ambientes permite realizar cambios y pruebas sin afectar la operación real

- **Desarrollo:** utilizado para construir y comprobar cambios durante la implementación. Aqui se realizan las pruebas unitarias.
- **Prueba:** utilizado para evaluar versiones identificadas con datos y condiciones controladas. Ambiente de testers donde se realizan las pruebas de sistema.
- **Preproducción o aceptación:** Ambiente semejante a produccion donde se realizan las pruebas de aceptacion, previo a la entrega final.
- **Producción:** Ambiente con la configuracion de HW y SW que tienen los usuarios finales que utilizan el sistema real. En este entorno no se realizan pruebas.

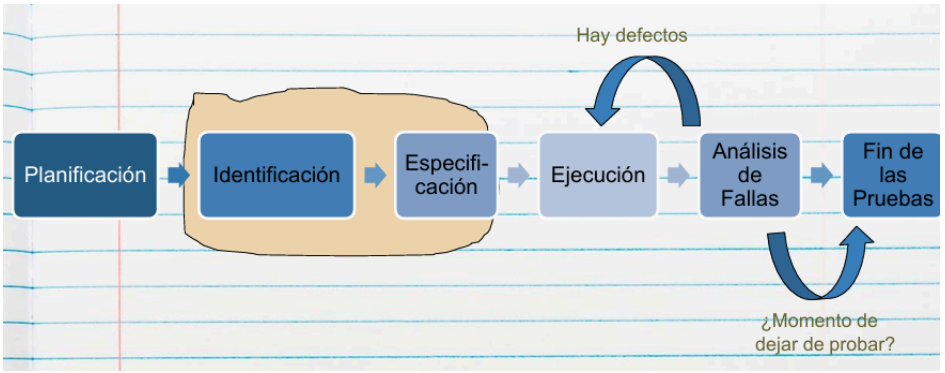
La distribución en tres o cuatro ambientes es una referencia presentada en clase, no una cantidad universal.Para interpretar una prueba deben conocerse la versión, configuración, datos y recursos utilizados. Esto es especialmente importante en pruebas de rendimiento, carga y compatibilidad, cuyos resultados dependen del entorno.

Proceso de pruebas

El Testing es un proceso definido (que se lleva a cabo durante todo el ciclo de vida del producto), por lo que está compuesto de etapas detalladas. Generalmente el proceso de pruebas es estándar.

- **Planificación:** En esta etapa se establece cómo se incluirá el testing en el proyecto y se realiza el plan de testing que entre otras cosas involucra riesgos y objetivos, estrategia, recursos y criterios de aceptación para finalizar las pruebas.
- **Diseño:** Mediante el plan de testing se identifican los datos necesarios, diseñan y priorizan los CP y el entorno de prueba. Se analiza si los requerimientos son testeables o no y si se usa regresión o no.

- **Ejecución:** Se ejecutan los CP registrando salidas y comparando con resultados esperados, incluyendo los defectos en el reporte de defectos. Se realiza en un entorno de prueba con los datos necesarios y se automatiza lo mas posible.
- **Analisis de fallas:** Se hace un seguimiento de la corrección de los defectos encontrados hasta que se cierren todos los CP, es decir que se hayan solucionado todos. Se evalúa el criterio de aceptación, se reporta el resultado de las pruebas a los interesados, se verifica los entregables y que los defectos se hayan corregido y se evalúa cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.
    - Acá se confecciona el informe de reportes.
- **Evaluación y cierre:** analizar defectos y cobertura, comprobar criterios de finalización, informar riesgos y conservar resultados y lecciones aprendidas.



Técnicas o Métodos para diseñar pruebas

Se utilizan para pasar por la mayor cantidad de funcionalidades con la menor cantidad de casos pruebas confeccionados, debido a que el tiempo y el presupuesto para diseñarlos y ejecutarlos es limitado. Hay dos grandes enfoques: Caja Negra y Caja Blanca. Cada uno tiene fortalezas y debilidades particulares: un método puede ser bueno para algunas cosas, y no para otras cosas. El mejor método es no usar un único método, sino usar una variedad de técnicas ayudará a un testing efectivo.

Caja negra

Diseña casos a partir del comportamiento esperado y las especificaciones, sin depender de la estructura interna. No tenemos acceso al proceso que transforma entradas en salida. Es dinámico.

Basados en especificaciones:

Son aquellas que se ejecutan usando las especificaciones realizadas del producto

Partición de equivalencias

Es una técnica de caja negra que divide los valores posibles de las **condiciones externas de entrada y salida** en **clases de equivalencia**, cuyos elementos se espera que reciban un tratamiento equivalente. Permite seleccionar valores representativos sin probar todas las posibilidades. El procedimiento es:

1. **Identificar las condiciones externas:** datos ingresados, selecciones, estados existentes y respuestas observables del sistema. Los campos independientes se analizan por separado, como *calle 1* y *calle 2*.
2. **Definir las clases válidas e inválidas:** deben cubrir los valores posibles de cada condición, sin superponerse dentro de una misma partición. Identificar las respuestas esperadas, incluidos los rechazos correspondientes a entradas inválidas.
3. **Diseñar los casos:** seleccionar valores concretos de las clases. Combinar varias clases válidas compatibles en un caso y, para comprobar una inválida, mantener las demás entradas válidas para evitar que un problema oculte otro.

- **Prioridad:** asignarla según el impacto y el riesgo. Los caminos principales y los errores críticos suelen tener prioridad alta; una validación también puede ser crítica.
- **Nombre:** describir claramente el escenario y qué se comprueba.
- **Precondiciones:** indicar el contexto necesario antes de ejecutar el caso, con datos concretos: usuario, permisos, estado del sistema y opciones disponibles.
- **Pasos:** detallar acciones claras y reproducibles, indicando los valores ingresados o seleccionados.
- **Resultados esperados:** especificar las respuestas que deben observarse y los datos correspondientes. Pueden vincularse con los pasos que las producen o agruparse al final; no toda acción requiere una salida visible.

Para aplicar valores límite, se seleccionan representantes en los extremos de las clases y sus vecinos, cuando existen límites definidos.

Análisis de valores límite

Complementa la partición de equivalencias seleccionando valores en las fronteras y sus vecinos, donde suelen aparecer errores de comparación o de cambio de tramo.Para un intervalo de números enteros válido entre **a y b inclusive**, se consideran: **$a - 1$, a, $a + 1$, $b - 1$, b, $b + 1$.**

La separación depende del dato: una unidad para enteros, un centavo para importes o un día para determinadas fechas. Debe respetarse si los extremos son inclusivos o exclusivos.

Basadas en la experiencia

La **adivinanza de defectos** utiliza conocimientos sobre errores frecuentes para orientar casos.

El **testing exploratorio** combina aprendizaje, diseño y ejecución: el tester investiga el producto y utiliza los resultados para decidir qué comprobar después. Puede organizarse mediante objetivos y sesiones con registro de hallazgos.

Caja blanca

Es un enfoque que diseña pruebas a partir del **conocimiento de la estructura interna y la lógica del código**. Los casos se seleccionan para comprobar distintos elementos:

- **Sentencias:** ejecutar las instrucciones del código.
- **Decisiones o ramas:** recorrer los resultados posibles de cada decisión, como verdadero y falso.
- **Condiciones:** evaluar como verdaderas y falsas las condiciones individuales de una decisión.
- **Caminos y bucles:** comprobar recorridos y repeticiones.

La **complejidad ciclomática** indica la cantidad de caminos linealmente independientes y ayuda a orientar las pruebas de caminos básicos.

Una cobertura del 100 % no garantiza ausencia de defectos: solo indica que se cubrió el criterio medido. A diferencia de caja negra, que se basa en especificaciones y comportamientos observables, **caja blanca utiliza la estructura del código**. No equivale a evaluación estática: sus pruebas pueden ejecutar el software. Las revisiones y el análisis estático lo examinan sin ejecutarlo.

Tipos de prueba

Los tipos de prueba se seleccionan según el aspecto que se necesita evaluar. Siguiendo la organización utilizada en clase, se agrupan según atiendan requerimientos funcionales o no funcionales.

Pruebas orientadas a requerimientos funcionales

Controla que el software se comporte de la misma manera que lo especificado en la documentación, cumpliendo con las funcionalidades y características definidas. Se basa en los requerimientos funcionales y el proceso de negocio. Hay testing funcional basado en dos aspectos:

- Basado en requerimientos: cuando se prueban requerimientos específicos, apunta a probar una funcionalidad sola (utilizan a los requisitos definidos en una ERS o los acuerdos que contienen las pruebas de usuario y los criterios de aceptación de una US para realizar las pruebas.)
- Basado en proceso de negocio: cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso. Por ejemplo, en una venta se prueba la búsqueda del artículo, la selección y facturación del mismo.

Pruebas orientadas a requerimientos no funcionales

- **Desempeño o performance:** evalúan tiempos de respuesta y utilización de recursos bajo condiciones determinadas.
- **Escala completa o carga:** comprueban el funcionamiento con la demanda y configuración comprometidas, como determinada cantidad de usuarios concurrentes.
- **Estrés:** someten al sistema a condiciones extremas o superiores a las previstas para observar sus límites, degradación y recuperación.
- **Usabilidad:** evalúan cuánto esfuerzo requiere alcanzar un objetivo, considerando comprensión, tiempo, pasos y experiencia de uso.
- **Accesibilidad:** comprueban que personas con distintas capacidades puedan percibir, comprender y operar el sistema.
- **Mantenimiento:** evalúan la facilidad para analizar, modificar y comprobar cambios.
- **Fiabilidad:** evalúan la consistencia del funcionamiento y la confianza que puede depositarse en sus resultados bajo las condiciones previstas.
- **Portabilidad:** comprueban la adaptación a los entornos requeridos, como sistemas operativos, navegadores o dispositivos.
- **Seguridad:** comprueban protección de información, autenticación y permisos. Cuando corresponda al sistema completo, también se consideran controles físicos.
- **Interfaz de usuario:** evalúan controles, mensajes, combinaciones, habilitación de opciones y correspondencia con las reglas de negocio. Pueden comprender aspectos funcionales y de usabilidad.

Smoke test: Primera corrida de los tests de sistema que provee cierto aseguramiento de que el software que está siendo probado no provoca una falla catastrófica. Se suele realizar en el ciclo 0.

Testing y ciclo de vida

En un **ciclo secuencial o en cascada**, la ejecución de pruebas sobre el sistema completo suele concentrarse después de la implementación. Sin embargo, **la planificación y el diseño de pruebas pueden comenzar desde los requisitos**, y las revisiones permiten detectar problemas tempranamente.

En un **ciclo iterativo e incremental**, las pruebas acompañan cada iteración y evalúan las funcionalidades nuevas y su integración con lo existente. Se utilizan **pruebas de regresión** para detectar si los cambios afectaron comportamientos que antes funcionaban. **Los problemas de integración pueden aparecer en ambos enfoques**; integrar y probar frecuentemente permite detectarlos antes.

Artefactos del testing:

- **Plan de pruebas:** establece el alcance, objetivos, riesgos, estrategia, recursos, responsables, calendario, ambientes, datos y herramientas. Define los criterios de finalización y el enfoque de regresión. Puede elaborarse antes de disponer del código, a partir de requisitos, casos de uso, historias de usuario y otras fuentes. Se coordina con la planificación del proyecto.
- **Casos de prueba:** describen qué se comprobará, con qué precondiciones y datos, mediante qué acciones y con qué resultados esperados. Se derivan de requisitos, información del cliente, reglas de negocio, especificaciones o código. Deben ser **claros y reproducibles**, y pueden reutilizarse mientras sigan siendo pertinentes, actualizándolos cuando cambie el producto.
- **Registros de ejecución:** identifican la versión y el ambiente evaluados, los casos ejecutados, los resultados obtenidos y su estado: exitoso, fallido o bloqueado.
- **Reporte de incidentes o defectos:** documenta un problema para facilitar su investigación y corrección. Incluye descripción, versión, entorno, precondiciones, datos, pasos, resultado esperado, resultado obtenido y evidencias. También puede indicar severidad y prioridad.
- **Informe final de pruebas:** resume el alcance evaluado, los resultados, la cobertura, los ciclos realizados, los defectos pendientes y los riesgos restantes. Permite valorar el cumplimiento de los criterios de finalización y apoyar decisiones sobre la entrega.

Testing en ambientes ágiles

Enfoque general

En ambientes ágiles, el testing se integra al desarrollo desde el comienzo. Las pruebas ayudan a **comprender necesidades, orientar la construcción y obtener retroalimentación rápida**. La calidad es responsabilidad de todo el equipo. Programadores, testers y representantes del negocio colaboran para definir ejemplos, construir, evaluar y mejorar el producto durante cada iteración.

El testing ágil conserva la necesidad de descubrir problemas y evaluar el producto. Cambia principalmente **cómo se organizan las actividades, cuándo se realizan y cómo colaboran las personas**.Las pruebas acompañan el desarrollo para prevenir problemas y reducir el tiempo de retroalimentación. El tester necesita comprender el negocio y contribuir a validar que el producto genere valor. Encontrar defectos es una contribución a la construcción de un mejor sistema. La automatización permite repetir comprobaciones y regresión con frecuencia dentro del tiempo disponible. Su beneficio depende de la relevancia y calidad de las pruebas: aumentar su cantidad no garantiza automáticamente mayor calidad.

Manifiesto del testing ágil

El manifiesto del testing ágil propone valorar:

- **Probar durante el desarrollo** sobre concentrar las pruebas al final.
- **Prevenir defectos** sobre limitarse a encontrarlos posteriormente.
- **Comprender lo que se prueba** sobre comprobar funcionalidades mecánicamente.
- **Construir un mejor producto** sobre considerar que encontrar fallas es el único objetivo.
- **La responsabilidad de todo el equipo** sobre delegar el testing exclusivamente al tester.

Estas prioridades orientan las pruebas hacia la colaboración, la prevención y la retroalimentación temprana. No eliminan las pruebas finales ni la búsqueda de defectos; las integran al trabajo continuo del equipo.

Principios del testing ágil

- Comenzar a probar tempranamente.
- Integrar el testing al trabajo continuo.
- Compartir la responsabilidad de las pruebas.
- Reducir el tiempo entre un cambio y su evaluación.
- Expresar expectativas mediante pruebas.
- Mantener el código limpio y corregir defectos rápidamente.
- Producir documentación útil y proporcional.
- Incluir las pruebas en la definición de terminado.
- Utilizar pruebas para orientar el desarrollo.

Rol del tester

El tester aporta conocimientos para aclarar requisitos, identificar riesgos, diseñar pruebas y explorar el comportamiento del producto. También ayuda a expresar necesidades del negocio mediante ejemplos verificables. Trabaja junto con los programadores y los interesados. La responsabilidad compartida permite utilizar especialidades sin crear una separación rígida entre quienes construyen y quienes evalúan.

Prácticas de testing ágil

- Pruebas unitarias y de integración automatizadas.
- Pruebas de regresión de sistema automatizadas.
- Testing exploratorio.
- TDD -> Explicado más adelante

- ATDD.
- Control de versiones de las pruebas junto con el código.

ATDD — Desarrollo conducido por pruebas de aceptación

Antes de implementar, representantes del negocio, testers y programadores acuerdan ejemplos que expresan el comportamiento esperado. Estos ejemplos se convierten en pruebas que orientan el desarrollo y permiten comprobar condiciones de satisfacción. TDD se concentra principalmente en el diseño y comportamiento de unidades pequeñas; ATDD ayuda a construir una comprensión compartida de las expectativas del negocio.

Cuadrantes de Agile Testing

Los cuadrantes de Crispin y Gregory combinan dos perspectivas: pruebas orientadas a tecnología o negocio, y pruebas que apoyan la construcción o evalúan críticamente el producto.

Cuadrante	Propósito	Ejemplos
Q1: tecnología, apoyo al equipo	Orientar y comprobar la construcción técnica.	Pruebas unitarias y de componentes.
Q2: negocio, apoyo al equipo	Aclarar y comprobar el comportamiento esperado.	Ejemplos, pruebas de historias, funcionales y prototipos.
Q3: negocio, evaluación del producto	Investigar si el producto satisface realmente al usuario.	Exploratorias, usabilidad y aceptación de usuario.
Q4: tecnología, evaluación del producto	Evaluar atributos y riesgos técnicos.	Rendimiento, carga, seguridad y otros atributos de calidad.

Los cuadrantes no son fases ni un orden de ejecución. Ayudan a reconocer qué evaluaciones se necesitan según los riesgos y objetivos del producto.

Pirámide de automatización

La pirámide orienta la distribución de las pruebas automatizadas:

- **Base:** muchas pruebas unitarias y de componentes, rápidas y focalizadas.



- **Nivel intermedio:** pruebas de servicios, API y aceptación por debajo de la interfaz.
- **Parte superior:** una cantidad menor de pruebas mediante la interfaz gráfica, generalmente más lentas y costosas de mantener.

Se busca evaluar cada comportamiento en el nivel que proporcione información útil con un costo razonable. Las pruebas manuales, exploratorias y de aceptación humana complementan esta estrategia.

Automatización, integración continua y Definition of Done

La automatización facilita ejecutar regresión con frecuencia, obtener resultados repetibles y detectar problemas poco después de introducir cambios. La integración continua combina integración frecuente, construcción y pruebas automáticas. La **Definition of Done** establece las condiciones para considerar terminado el trabajo. Puede exigir pruebas exitosas, revisión, integración y cumplimiento de criterios de calidad. Las actividades necesarias para alcanzar esa definición deben realizarse dentro del trabajo del equipo y de la iteración.

Desafíos culturales

La transición requiere dejar de tratar los defectos como un conflicto entre personas y utilizarlos como información para mejorar. También requiere aprender a colaborar, compartir conocimientos y comunicar resultados y riesgos. La independencia de criterio sigue siendo valiosa, pero se combina con colaboración y responsabilidad colectiva. Las revisiones, auditorías y modelos de calidad pueden adaptarse a esta forma de trabajo.

Testing en agil vs en tradicional

En el **enfoque tradicional**, las pruebas suelen organizarse como una actividad diferenciada del desarrollo, con planificación, documentación y responsabilidades especializadas. En un ciclo secuencial, la ejecución de pruebas sobre el sistema integrado se concentra después de la implementación. Sin embargo, **la planificación y el diseño de pruebas pueden comenzar desde los requisitos**, y las revisiones permiten detectar problemas antes de ejecutar el software.

En el **enfoque ágil**, el testing se integra al desarrollo de cada incremento. Testers, desarrolladores y representantes del negocio colaboran desde el comienzo para aclarar necesidades y definir ejemplos y criterios de aceptación. Las pruebas acompañan la construcción y pueden orientarla mediante prácticas como **TDD y ATDD**. La calidad es responsabilidad de todo el equipo, aunque existan personas especializadas en testing.

La diferencia principal está en **cómo se organiza el trabajo y cuándo se obtiene retroalimentación**. El enfoque tradicional tiende a apoyarse más en planes, documentación y transferencia entre funciones;

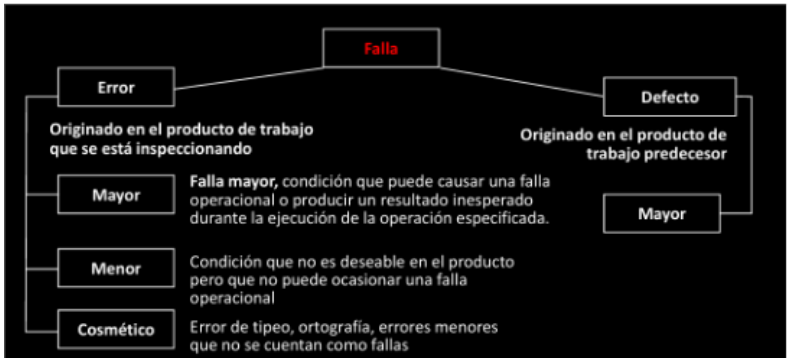
Agile enfatiza la colaboración continua, las pruebas tempranas y la adaptación. La documentación sigue siendo necesaria, pero se busca que sea útil y proporcional al contexto.

La **automatización puede utilizarse en ambos enfoques**, pero cobra especial importancia en Agile para ejecutar pruebas frecuentes y regresión con cada cambio. Se complementa con pruebas exploratorias y evaluaciones humanas. En ambos casos se busca detectar defectos y aportar información sobre la calidad; **Agile no elimina la planificación ni el enfoque tradicional exige dejar toda evaluación para el final**.

Revisiones técnicas

Verificación y validación

La verificación y validación —V&V— comprende actividades planificadas durante todo el ciclo de vida. La verificación comprueba el cumplimiento de requisitos y especificaciones; la validación comprueba la satisfacción de las necesidades del usuario. Las revisiones técnicas contribuyen a V&V mediante el examen estático de productos de trabajo, permitiendo detectar problemas tempranamente y reducir el retrabajo.



¿Qué es una revisión técnica?

Una revisión técnica examina un producto de trabajo para detectar defectos en etapas tempranas del desarrollo. Puede aplicarse a requisitos, diseños, código, planes y casos de prueba. Es una actividad **estática**: no requiere ejecutar el programa. Por eso permite encontrar problemas antes de disponer de software funcionando y complementa las pruebas dinámicas. Se realiza de a pares.

Tipos de revisión

INFORMAL - Walkthrough: el autor guía a otros participantes por el producto; estos realizan preguntas y comentarios sobre posibles errores o incumplimientos. En la clasificación utilizada en clase, es informal, tiene poca sobrecarga y favorece la comprensión y capacitación.

FORMAL - Inspección: revisión formal orientada a detectar defectos y comprobar requisitos y estándares. Utiliza roles definidos, preparación individual, checklists, registro de hallazgos, métricas y seguimiento.

Roles de una inspección

- **Autor:** crea o mantiene el producto y realiza las correcciones.
- **Moderador:** planifica, coordina y verifica el seguimiento.
- **Lector:** presenta o recorre el material.
- **Anotador:** registra hallazgos.
- **Inspectores:** estudian el producto y buscan problemas.

La distribución concreta puede variar según el tamaño del equipo y el proceso utilizado.

Proceso de inspección

1. **Planificación:** seleccionar material, participantes y criterios.
2. **Presentación general:** explicar el contexto cuando sea necesario.
3. **Preparación individual:** estudiar el material antes de la reunión.
4. **Reunión:** reunir, discutir y registrar los defectos.
5. **Corrección:** el autor trata los problemas.
6. **Seguimiento:** comprobar las correcciones y decidir si hace falta reinspeccionar.

Durante la revisión se debe **evaluar el producto y mantener el foco en los problemas**, evitando convertir la reunión en una evaluación personal o en una discusión extensa sobre todas las soluciones posibles.

Clasificación de hallazgos

Los problemas encontrados durante una inspección pueden clasificarse en:

- **Mayores:** pueden provocar fallas operacionales o resultados incorrectos. Por ejemplo, una regla de negocio mal especificada.
- **Menores:** afectan la claridad, el mantenimiento o el cumplimiento de estándares, sin provocar directamente una falla operacional. Por ejemplo, nombres de variables que incumplen las convenciones acordadas.
- **Cosméticos:** problemas de presentación que no alteran el significado ni el funcionamiento, como errores tipográficos sin consecuencias.

La clasificación depende del impacto del problema: un error de escritura que modifica el sentido de un requisito puede ser mayor.

Resultados de la inspección

La inspección puede concluir en **aceptar el producto sin modificaciones**, **aceptarlo provisionalmente sujeto a correcciones** o **rechazarlo y solicitar correcciones y una nueva inspección**. Los hallazgos y las decisiones se registran en un informe, y el seguimiento verifica que las correcciones se hayan realizado.

Ventajas, dificultades y herramientas

Las revisiones detectan defectos tempranamente, pueden aplicarse a productos incompletos y favorecen el conocimiento compartido. Requieren preparación, capacitación y tiempo; sin una buena organización pueden convertirse en reuniones poco efectivas. Se apoyan en checklists, guías de lectura, estándares, plantillas y registros de hallazgos. Las herramientas de revisión y análisis estático pueden ayudar a encontrar problemas, pero sus resultados necesitan interpretación. Pueden medirse densidad de defectos, esfuerzo de revisión, esfuerzo por defecto, correcciones y re inspecciones. Las métricas deben utilizarse para mejorar la revisión y el proceso.

Métricas de inspección

Permiten evaluar la efectividad y el costo de las inspecciones y mejorar el proceso:

- **Densidad de defectos:** cantidad de defectos encontrados / tamaño del material revisado.
- **Esfuerzo de inspección:** suma del esfuerzo de planificación, preparación, reunión y retrabajo.
- **Esfuerzo por defecto:** esfuerzo de inspección / cantidad de defectos encontrados.
- **Porcentaje de reinspecciones:** cantidad de reinspecciones / cantidad de inspecciones × 100.

TDD — Desarrollo conducido por pruebas

TDD (*Test-Driven Development*) es una práctica de desarrollo que combina escribir las pruebas primero y refactorizar el código. Se trabaja en pasos pequeños: antes de implementar un comportamiento, se escribe una prueba automatizada —generalmente unitaria— que expresa lo esperado. Las pruebas orientan el diseño y proporcionan retroalimentación inmediata.

Ciclo Red–Green–Refactor

El corazón de TDD consiste en repetir tres pasos:

1. **Red — Rojo:** escribir una prueba para un comportamiento pequeño y comprobar que falla porque todavía no está implementado.
2. **Green — Verde:** escribir el código mínimo necesario para que la prueba pase y comprobar que las anteriores siguen pasando.
3. **Refactor:** mejorar la estructura del código, eliminando duplicación y complejidad innecesaria, sin cambiar su comportamiento observable.

Después se agrega una nueva prueba y se repite el ciclo. Así, la funcionalidad y el diseño evolucionan gradualmente.

Las tres leyes de TDD

- No escribir código de producción sin una prueba que falle.
- No escribir más código de prueba del necesario para producir ese fallo.
- No escribir más código de producción del necesario para hacer pasar la prueba.

Patrones principales

Los patrones ayudan a decidir **qué prueba escribir y cómo avanzar hacia una solución**.

Para comenzar o ampliar las pruebas, se utilizan casos pequeños, pruebas iniciales que exploran el comportamiento esperado y nuevos casos que cubren variantes. Una **prueba de regresión** reproduce un defecto descubierto para detectar si reaparece.

Para alcanzar el estado verde, la filmina destaca:

- **Fake It:** utilizar temporalmente una respuesta fija o una solución muy simple para hacer pasar la prueba.
- **Triangulación:** agregar casos con datos diferentes que obliguen a generalizar la solución.
- **Implementación obvia:** escribir directamente la solución cuando es clara y sencilla.

Refactorización

Refactorizar consiste en **mejorar la estructura interna del software sin modificar su comportamiento observable**. Permite eliminar duplicación, simplificar el código, mejorar nombres y facilitar futuros cambios. Se parte de pruebas exitosas, se realizan modificaciones pequeñas y se vuelven a ejecutar las pruebas. Esto ayuda a incorporar lo aprendido durante el desarrollo y evitar el deterioro del diseño.

Beneficios y alcance

TDD favorece código comprensible, componentes con **alta cohesión y bajo acoplamiento**, detección temprana de problemas y mayor confianza para modificar y mantener el software. Escribir primero las pruebas también obliga a pensar cómo se utilizará el componente.

Sin embargo, las pruebas exitosas **no garantizan por sí solas la calidad completa del producto**. TDD debe complementarse con pruebas de integración, aceptación, seguridad, rendimiento y otras evaluaciones necesarias según el contexto.